stack and is scheduled independently of the other threads. When a thread makes a blocking system call, the other threads in the same address space are not affected. Threads packages can be implemented in either user space or by the kernel, but there are problems to be solved either way. The use of lightweight threads has led to some interesting results in lightweight RPC as well. Pop-up threads are also an important technique.

Two models of organizing the processors are commonly used: the workstation model and the processor pool model. In the former, each user has his own workstation, sometimes with the ability to run processes on idle workstations. In the latter, the entire computing facility is a shared resource. Processors are then allocated dynamically to users as needed and returned to the pool when the work is done. Hybrid models are also possible.

Given a collection of processors, some algorithm is needed for assigning processes to processors. Such algorithms can be deterministic or heuristic, centralized or distributed, optimal or suboptimal, local or global, and sender-initiated or receiver-initiated.

Although processes are normally scheduled independently, using coscheduling, to make sure that processes that must communicate are running at the same time, performance can be improved.

Fault tolerance is important in many distributed systems. It can be achieved using triple modular redundancy, active replication, or primary-backup replication. The two-army problem cannot be solved in the presence of unreliable communication, but the Byzantine generals problem can be solved if more than two-thirds of the processors are nonfaulty.

Finally, real-time distributed systems are also important. They come in two varieties: soft real time and hard real time. Event-triggered systems are interrupt driven, whereas time-triggered systems sample the external devices at fixed intervals. Real-time communication must use predictable protocols, such as token rings or TDMA. Both dynamic and static scheduling of tasks is possible. Dynamic scheduling happens at run time; static scheduling happens in advance.

PROBLEMS

1. In this problem you are to compare reading a file using a single-threaded file server and a multithreaded server. It takes 15 msec to get a request for work, dispatch it, and do the rest of the necessary processing, assuming that the data needed are in the block cache. If a disk operation is needed, as is the case one-third of the time, an additional 75 msec is required, during which time the thread sleeps. How many requests/sec can the server handle if it is single threaded? If it is multithreaded?

2. In Fig. 4-3 the register set is listed as a per-thread rather than a per-process item. Why? After all, the machine has only one set of registers.

3. In the text, we described a multithreaded file server, showing why it is better than a single-threaded server and a finite-state machine server. Are there any circumstances in which a single-threaded server might be better? Give an example.

4. In the discussion on global variables in threads, we used a procedure *create_global* to allocate storage for a pointer to the variable, rather than the variable itself. Is this essential, or could the procedures work with the values themselves just as well?

5. Consider a system in which threads are implemented entirely in user space, with the runtime system getting a clock interrupt once a second. Suppose that a clock interrupt occurs while some thread is executing in the runtime system. What problem might occur? Can you suggest a way to solve it?

6. Suppose that an operating system does not have anything like the SELECT system call to see in advance if it is safe to read from a file, pipe, or device, but it does allow alarm clocks to be set that interrupt blocked system calls. Is it possible to implement a threads package in user space under these conditions? Discuss.

7. In a certain workstation-based system, the workstations have local disks that hold the system binaries. When a new binary is released, it is sent to each workstation. However, some workstations may be down (or switched off) when this happens. Devise an algorithm that allows the updating to be done automatically, even though workstations are occasionally down.

8. Can you think of any other kinds of files that can safely be stored on user workstations of the type described in the preceding problem?

9. Would the scheme of Bershad et al. to make local RPCs go faster also work in a system with only one thread per process? How about with Peregrine?

10. When two users examine the registry in Fig. 4-12 simultaneously, they may accidentally pick the same idle workstation. How can the algorithm be subtly changed to prevent this race?

11. Imagine that a process is running remotely on a previously idle workstation, which, like all the workstations, is diskless. For each of the following UNIX system calls, tell whether it has to be forwarded back to the home machine:

    (a) READ (get data from a file).
    (b) IOCTL (change the mode of the controlling terminal).
    (c) GETPID (return the process id).

12. Compute the response ratios for Fig. 4-15 using processor 1 as the benchmark processor. Which assignment minimizes the average response ratio?

13. In the discussion of processor allocation algorithms, we pointed out that one choice is between centralized and distributed and another is between optimal and suboptimal. Devise two optimal location algorithms, one centralized and one decentralized.

14. In Fig. 4-17 we see two different allocation schemes, with different amounts of network traffic. Are there any other allocations that are better still? Assume that no machine may run more than four processes.

15. The up-down algorithm described in the text is a centralized algorithm design to allocate processors fairly. Invent a centralized algorithm whose goal is not fairness but distributing the load uniformly.

16. When a certain distributed system is overloaded, it makes m attempts to find an idle workstation to offload work to. The probability of a workstation having k jobs is given by the Poisson formula

$$P(k) = \frac{\lambda^k e^{-\lambda}}{k!}$$

where λ is the mean number of jobs per workstation. What is the probability that an overloaded workstation finds an idle one (i.e., one with $k = 0$) in m attempts? Evaluate your answer numerically for $m = 3$ and values of λ from 1 to 4.

17. Using the data of Fig. 4-20, what is the longest UNIX pipeline that can be co-scheduled?

18. Can the model of triple modular redundancy described in the text handle Byzantine failures?

19. How many failed elements (devices plus voters) can Fig. 4-21 handle? Given an example of the worst case that can be masked.

20. Does TMR generalize to five elements per group instead of three? If so, what properties does it have?

21. Eloise lives at the Plaza Hotel. Her favorite activity is standing in the main lobby pushing the elevator. She can do this over and hour, for hours on end. The Plaza is installing a new elevator system. They can choose between a time-triggered system and an event-triggered system. Which one should they choose? Discuss.

22. A real-time system has periodic processes with the following computational requirements and periods:

P1: 20 msec every 40 msec
P2: 60 msec every 500 msec
P3: 5 msec every 20 msec
P4: 15 msec every 100 msec

Is this system schedulable on one CPU?

23. Is it possible to determine the priorities that the rate monotonic algorithm would assign to the processes in the preceding problem? If so, what are they? If not, what information is lacking?

24. A network consists of two parallel wires: the forward link, on which packets travel from left to right, and the reverse link, on which they travel from right to left. A generator at the head of each wire generates a continuous stream of packet frames, each holding an empty/full bit (initially empty). All the computers are located between the two wires, attached to both. To send a packet, a computer determines which wire to use (depending on whether the destination is to the left or right of it), waits for an empty frame, puts a packet in it, and marks the frame as full. Does this network satisfy the requirements for a real-time system? Explain.

25. The assignment of processes to slots in Fig. 4-32 is arbitrary. Other assignments are also possible. Find an alternative assignment that improves the performance of the second example.

5

Distributed File Systems

A key component of any distributed system is the file system. As in single processor systems, in distributed systems the job of the file system is to store programs and data and make them available as needed. Many aspects of distributed file systems are similar to conventional file systems, so we will not repeat that material here. Instead, we will concentrate on those aspects of distributed file systems that are different from centralized ones.

To start with, in a distributed system, it is important to distinguish between the concepts of the file service and the file server. The **file service** is the specification of what the file system offers to its clients. It describes the primitives available, what parameters they take, and what actions they perform. To the clients, the file service defines precisely what service they can count on, but says nothing about how it is implemented. In effect, the file service specifies the file system's interface to the clients.

A **file server**, in contrast, is a process that runs on some machine and helps implement the file service. A system may have one file server or several. In particular, they should not know how many file servers there are and what the location or function of each one is. All they know is that when they call the procedures specified in the file service, the required work is performed somehow, and the required results are returned. In fact, the clients should not even know that the file service is distributed. Ideally, it should look the same as a normal single-processor file system.

Since a file server is normally just a user process (or sometimes a kernel process) running on some machine, a system may contain multiple file servers, each offering a different file service. For example, a distributed system may have two servers that offer UNIX file service and MS-DOS file service, respectively, with each user process using the one appropriate for it. In that way, it is possible to have a terminal with multiple windows, with UNIX programs running in some windows and MS-DOS programs running in other windows, with no conflicts. Whether the servers offer specific file services, such as UNIX or MS-DOS, or more general file services is up to the system designers. The type and number of file services available may even change as the system evolves.

5.1. DISTRIBUTED FILE SYSTEM DESIGN

A distributed file system typically has two reasonably distinct components: the true file service and the directory service. The former is concerned with the operations on individual files, such as reading, writing, and appending, whereas the latter is concerned with creating and managing directories, adding and deleting files from directories, and so on. In this section we will discuss the true file service interface; in the next one we will discuss the directory service interface.

5.1.1. The File Service Interface

For any file service, whether for a single processor or for a distributed system, the most fundamental issue is: What is a file? In many systems, such as UNIX and MS-DOS, a file is an uninterpreted sequence of bytes. The meaning and structure of the information in the files is entirely up to the application programs; the operating system is not interested.

On mainframes, however, many types of files exist, each with different properties. A file can be structured as a sequence of records, for example, with operating system calls to read or write a particular record. The record can usually be specified by giving either its record number (i.e., position within the file) or the value of some field. In the latter case, the operating system either maintains the file as a B-tree or other suitable data structure, or uses hash tables to locate records quickly. Since most distributed systems are intended for UNIX or MS-DOS environments, most file servers support the notion of a file as a sequence of bytes rather than as a sequence of keyed records.

A files can have **attributes**, which are pieces of information about the file but which are not part of the file itself. Typical attributes are the owner, size, creation date, and access permissions. The file service usually provides primitives to read and write some of the attributes. For example, it may be possible to change the access permissions but not the size (other than by appending data to

the file). In a few advanced systems, it may be possible to create and manipulate user-defined attributes in addition to the standard ones.

Another important aspect of the file model is whether files can be modified after they have been created. Normally, they can be, but in some distributed systems, the only file operations are CREATE and READ. Once a file has been created, it cannot be changed. Such a file is said to be **immutable**. Having files be immutable makes it much easier to support file caching and replication because it eliminates all the problems associated with having to update all copies of a file whenever it changes.

Protection in distributed systems uses essentially the same techniques as in single-processor systems: capabilities and access control lists. With capabilities, each user has a kind of ticket, called a **capability**, for each object to which it has access. The capability specifies which kinds of accesses are permitted (e.g., reading is allowed but writing is not).

All **access control list** schemes associate with each file a list of users who may access the file and how. The UNIX scheme, with bits for controlling reading, writing, and executing each file separately for the owner, the owner's group, and everyone else is a simplified access control list.

File services can be split into two types, depending on whether they support an upload/download model or a remote access model. In the **upload/download model**, shown in Fig. 5-1(a), the file service provides only two major operations: read file and write file. The former operation transfers an entire file from one of the file servers to the requesting client. The latter operation transfers an entire file the other way, from client to server. Thus the conceptual model is moving whole files in either direction. The files can be stored in memory or on a local disk, as needed.

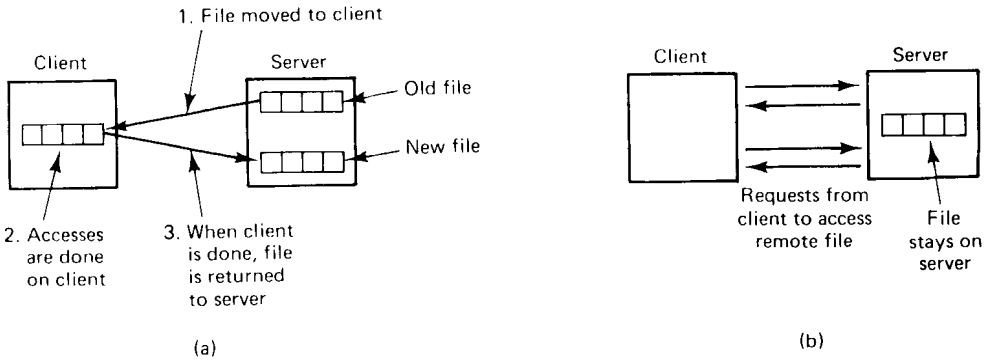


Fig. 5-1. (a) The upload/download model. (b) The remote access model.

The advantage of the upload/download model is its conceptual simplicity. Application programs fetch the files they need, then use them locally. Any

modified files or newly created files are written back when the program finishes. No complicated file service interface has to be mastered to use this model. Furthermore, whole file transfer is highly efficient. However, enough storage must be available on the client to store all the files required. Furthermore, if only a fraction of a file is needed, moving the entire file is wasteful.

The other kind of file service is the **remote access model**, as illustrated in Fig. 5-1(b). In this model, the file service provides a large number of operations for opening and closing files, reading and writing parts of files, moving around within files (LSEEK), examining and changing file attributes, and so on. Whereas in the upload/download model, the file service merely provides physical storage and transfer, here the file system runs on the servers, not on the clients. It has the advantage of not requiring much space on the clients, as well as eliminating the need to pull in entire files when only small pieces are needed.

5.1.2. The Directory Server Interface

The other part of the file service is the directory service, which provides operations for creating and deleting directories, naming and renaming files, and moving them from one directory to another. The nature of the directory service does not depend on whether individual files are transferred in their entirety or accessed remotely.

The directory service defines some alphabet and syntax for composing file (and directory) names. File names can typically be from 1 to some maximum number of letters, numbers, and certain special characters. Some systems divide file names into two parts, usually separated by a period, such as *prog.c* for a C program or *man.txt* for a text file. The second part of the name, called the **file extension**, identifies the file type. Other systems use an explicit attribute for this purpose, instead of tacking an extension onto the name.

All distributed systems allow directories to contain subdirectories, to make it possible for users to group related files together. Accordingly, operations are provided for creating and deleting directories as well as entering, removing, and looking up files in them. Normally, each subdirectory contains all the files for one project, such as a large program or document (e.g., a book). When the (sub)directory is listed, only the relevant files are shown; unrelated files are in other (sub)directories and do not clutter the listing. Subdirectories can contain their own subdirectories, and so on, leading to a tree of directories, often called a **hierarchical file system**. Figure 5-2(a) illustrates a tree with five directories

In some systems, it is possible to create links or pointers to an arbitrary directory. These can be put in any directory, making it possible to build not only trees, but arbitrary directory graphs, which are more powerful. The distinction between trees and graphs is especially important in a distributed system.

The nature of the difficulty can be seen by looking at the directory graph of

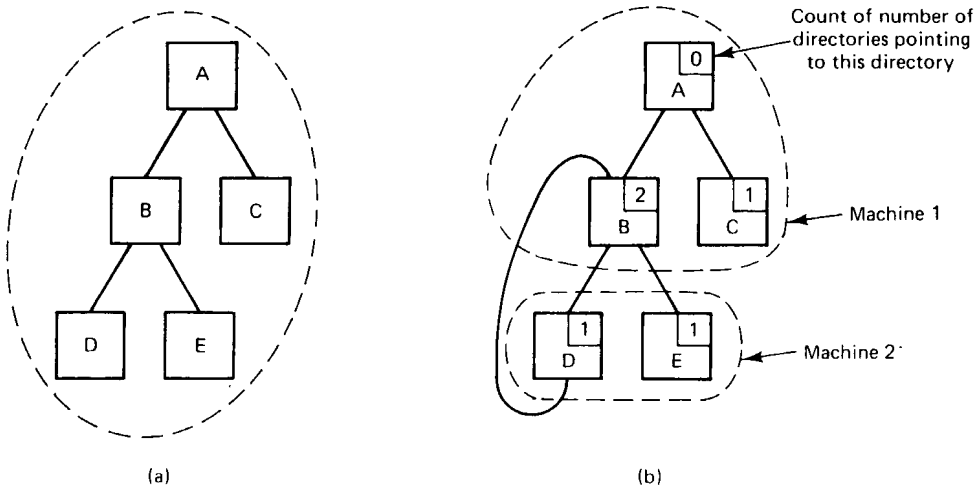


Fig. 5-2. (a) A directory tree contained on one machine. (b) A directory graph on two machines.

Fig. 5-2(b). In this figure, directory *D* has a link to directory *B*. The problem occurs when the link from *A* to *B* is removed. In a tree-structured hierarchy, a link to a directory can be removed only when the directory pointed to is empty. In a graph, it is allowed to remove a link to a directory as long as at least one other link exists. By keeping a reference count, shown in the upper right-hand corner of each directory in Fig. 5-2(b), it can be determined when the link being removed is the last one.

After the link from *A* to *B* is removed, *B*'s reference count is reduced from 2 to 1, which on paper is fine. However, *B* is now unreachable from the root of the file system (*A*). The three directories, *B*, *D*, and *E*, and all their files are effectively orphans.

This problem exists in centralized systems as well, but it is more serious in distributed ones. If everything is on one machine, it is possible, albeit somewhat expensive, to discover orphaned directories, because all the information is in one place. All file activity can be stopped and the graph traversed starting at the root, marking all reachable directories. At the end of this process, all unmarked directories are known to be unreachable. In a distributed system, multiple machines are involved and all activity cannot be stopped, so getting a "snapshot" is difficult, if not impossible.

A key issue in the design of any distributed file system is whether or not all machines (and processes) should have exactly the same view of the directory hierarchy. As an example of what we mean by this remark, consider Fig. 5-3. In Fig. 5-3(a) we show two file servers, each holding three directories and some

files. In Fig. 5-3(b) we have a system in which all clients (and other machines) have the same view of the distributed file system. If the path */D/E/x* is valid on one machine, it is valid on all of them.

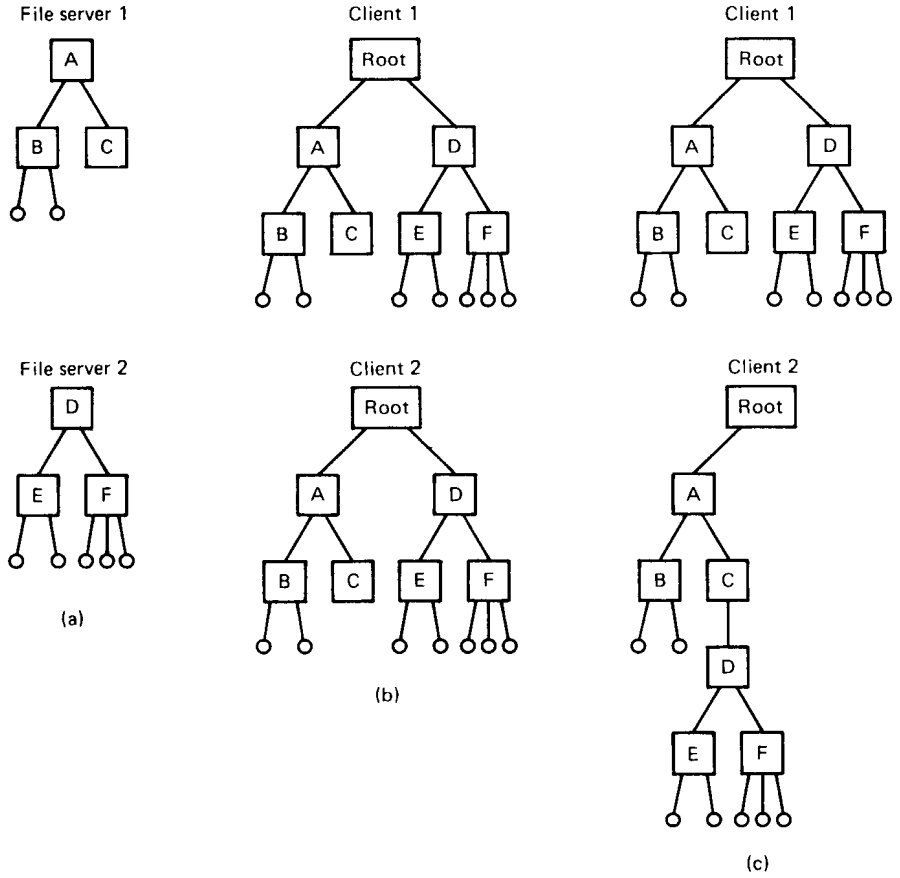


Fig. 5-3. (a) Two file servers. The squares are directories and the circles are files. (b) A system in which all clients have the same view of the file system. (c) A system in which different clients may have different views of the file system.

In contrast, in Fig. 5-3(c), different machines can have different views of the file system. To repeat the preceding example, the path */D/E/x* might well be valid on client 1 but not on client 2. In systems that manage multiple file servers by remote mounting, Fig. 5-3(c) is the norm. It is flexible and straightforward to implement, but it has the disadvantage of not making the entire system behave like a single old-fashioned timesharing system. In a timesharing system, the file

system looks the same to any process [i.e., the model of Fig. 5-3(b)]. This property makes a system easier to program and understand.

A closely related question is whether or not there is a global root directory, which all machines recognize as the root. One way to have a global root directory is to have this root contain one entry for each server and nothing else. Under these circumstances, paths take the form */server/path*, which has its own disadvantages, but at least is the same everywhere in the system.

Naming Transparency

The principal problem with this form of naming is that it is not fully transparent. Two forms of transparency are relevant in this context and are worth distinguishing. The first one, **location transparency**, means that the path name gives no hint as to where the file (or other object) is located. A path like */server1/dir1/dir2/x* tells everyone that *x* is located on server 1, but it does not tell where that server is located. The server is free to move anywhere it wants to in the network without the path name having to be changed. Thus this system has location transparency.

However, suppose that file *x* is extremely large and space is tight on server 1. Furthermore, suppose that there is plenty of room on server 2. The system might well like to move *x* to server 2 automatically. Unfortunately, when the first component of all path names is the server, the system cannot move the file to the other server automatically, even if *dir1* and *dir2* exist on both servers. The problem is that moving the file automatically changes its path name from */server1/dir1/dir2/x* to */server2/dir1/dir2/x*. Programs that have the former string built into them will cease to work if the path changes. A system in which files can be moved without their names changing is said to have **location independence**. A distributed system that embeds machine or server names in path names clearly is not location independent. One based on remote mounting is not either, since it is not possible to move a file from one file group (the unit of mounting) to another and still be able to use the old path name. Location independence is not easy to achieve, but it is a desirable property to have in a distributed system.

To summarize what we have said earlier, there are three common approaches to file and directory naming in a distributed system:

1. Machine + path naming, such as */machine/path* or *machine:path*.
2. Mounting remote file systems onto the local file hierarchy.
3. A single name space that looks the same on all machines.

The first two are easy to implement, especially as a way to connect up existing

systems that were not designed for distributed use. The latter is difficult and requires careful design, but it is needed if the goal of making the distributed system act like a single computer is to be achieved.

Two-Level Naming

Most distributed systems use some form of two-level naming. Files (and other objects) have **symbolic names** such as *prog.c*, for use by people, but they can also have internal, **binary names** for use by the system itself. What directories in fact really do is provide a mapping between these two naming levels. It is convenient for people and programs to use symbolic (ASCII) names, but for use within the system itself, these names are too long and cumbersome. Thus when a user opens a file or otherwise references a symbolic name, the system immediately looks up the symbolic name in the appropriate directory to get the binary name that will be used to locate the file. Sometimes the binary names are visible to the users and sometimes they are not.

The nature of the binary name varies considerably from system to system. In a system consisting of multiple file servers, each of which is self-contained (i.e., does not hold any references to directories or files on other file servers), the binary name can just be a local i-node number, as in UNIX.

A more general naming scheme is to have the binary name indicate both a server and a specific file on that server. This approach allows a directory on one server to hold a file on a different server. An alternative way to do the same thing that is sometimes preferred is to use a **symbolic link**. A symbolic link is a directory entry that maps onto a (server, file name) string, which can be looked up on the server named to find the binary name. The symbolic link itself is just a path name.

Yet another idea is to use capabilities as the binary names. In this method, looking up an ASCII name yields a capability, which can take one of many forms. For example, it can contain a physical or logical machine number or network address of the appropriate server, as well as a number indicating which specific file is required. A physical address can be used to send a message to the server without further interpretation. A logical address can be located either by broadcasting or by looking it up on a name server.

One last twist that is sometimes present in a distributed system but rarely in a centralized one is the possibility of looking up an ASCII name and getting not *one* but *several* binary names (i-nodes, capabilities, or something else). These would typically represent the original file and all its backups. Armed with multiple binary names, it is then possible to try to locate one of the corresponding files, and if that one is unavailable for any reason, to try one of the others. This method provides a degree of fault tolerance through redundancy.

5.1.3. Semantics of File Sharing

When two or more users share the same file, it is necessary to define the semantics of reading and writing precisely to avoid problems. In single-processor systems that permit processes to share files, such as UNIX, the semantics normally state that when a READ operation follows a WRITE operation, the READ returns the value just written, as shown in Fig. 5-4(a). Similarly, when two WRITEs happen in quick succession, followed by a READ, the value read is the value stored by the last write. In effect, the system enforces an absolute time ordering on all operations and always returns the most recent value. We will refer to this model as **UNIX semantics**. This model is easy to understand and straightforward to implement.

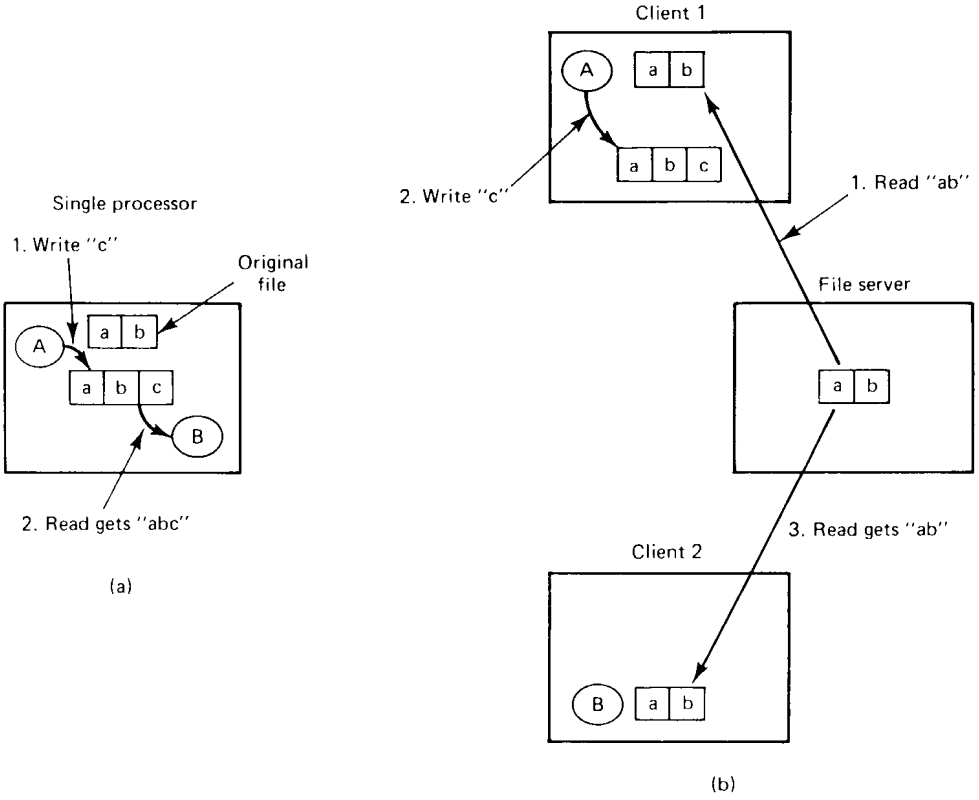


Fig. 5-4. (a) On a single processor, when a READ follows a WRITE, the value returned by the READ is the value just written. (b) In a distributed system with caching, obsolete values may be returned.

In a distributed system, UNIX semantics can be achieved easily as long as

there is only one file server and clients do not cache files. All READs and WRITEs go directly to the file server, which processes them strictly sequentially. This approach gives UNIX semantics (except for the minor problem that network delays may cause a READ that occurred a microsecond after a WRITE to arrive at the server first and thus get the old value).

In practice, however, the performance of a distributed system in which all file requests must go to a single server is frequently poor. This problem is often solved by allowing clients to maintain local copies of heavily used files in their private caches. Although we will discuss the details of file caching below, for the moment it is sufficient to point out that if a client locally modifies a cached file and shortly thereafter another client reads the file from the server, the second client will get an obsolete file, as illustrated in Fig. 5-4(b).

One way out of this difficulty is to propagate all changes to cached files back to the server immediately. Although conceptually simple, this approach is inefficient. An alternative solution is to relax the semantics of file sharing. Instead of requiring a READ to see the effects of all previous WRITEs, one can have a new rule that says: "Changes to an open file are initially visible only to the process (or possibly machine) that modified the file. Only when the file is closed are the changes made visible to other processes (or machines)." The adoption of such a rule does not change what happens in Fig. 5-4(b), but it does redefine the actual behavior (*B* getting the original value of the file) as being the correct one. When *A* closes the file, it sends a copy to the server, so that subsequent READs get the new value, as required. This rule is widely implemented and is known as **session semantics**.

Using session semantics raises the question of what happens if two or more clients are simultaneously caching and modifying the same file. One solution is to say that as each file is closed in turn, its value is sent back to the server, so the final result depends on who closes last. A less pleasant, but slightly easier to implement alternative is to say that the final result is one of the candidates, but leave the choice of which one unspecified.

One final difficulty with using caching and session semantics is that it violates another aspect of the UNIX semantics in addition to not having all READs return the value most recently written. In UNIX, associated with each open file is a pointer that indicates the current position in the file. READs take data starting at this position and WRITEs deposit data there. This pointer is shared between the process that opened the file and all its children. With session semantics, when the children run on different machines, this sharing cannot be achieved.

To see what the consequences of having to abandon shared file pointers are, consider a command like

```
run >out
```

where *run* is a shell script that executes two programs, *a* and *b*, one after

another. If both programs produce output, it is expected that the output produced by *b* will directly follow the output from *a* within *out*. The way this is achieved is that when *b* starts up, it inherits the file pointer from *a*, which is shared by the shell and both processes. In this way, the first byte that *b* writes directly follows the last byte written by *a*. With session semantics and no shared file pointers, a completely different mechanism is needed to make shell scripts and similar constructions that use shared file pointers work. Since no general-purpose solution to this problem is known, each system must deal with it in an ad hoc way.

A completely different approach to the semantics of file sharing in a distributed system is to make all files immutable. There is thus no way to open a file for writing. In effect, the only operations on files are CREATE and READ.

What is possible is to create an entirely new file and enter it into the directory system under the name of a previous existing file, which now becomes inaccessible (at least under that name). Thus although it becomes impossible to modify the file *x*, it remains possible to replace *x* by a new file atomically. In other words, although *files* cannot be updated, *directories* can be. Once we have decided that files cannot be changed at all, the problem of how to deal with two processes, one of which is writing on a file and the other of which is reading it, simply disappears.

What does remain is the problem of what happens when two processes try to replace the same file at the same time. As with session semantics, the best solution here seems to be to allow one of the new files to replace the old one, either the last one or nondeterministically.

A somewhat stickier problem is what to do if a file is replaced while another process is busy reading it. One solution is to somehow arrange for the reader to continue using the old file, even if it is no longer in any directory, analogous to the way UNIX allows a process that has a file open to continue using it, even after it has been deleted from all directories. Another solution is to detect that the file has changed and make subsequent attempts to read from it fail.

A fourth way to deal with shared files in a distributed system is to use atomic transactions, as we discussed in detail in Chap. 3. To summarize briefly, to access a file or a group of files, a process first executes some type of BEGIN TRANSACTION primitive to signal that what follows must be executed indivisibly. Then come system calls to read and write one or more files. When the work has been completed, an END TRANSACTION primitive is executed. The key property of this method is that the system guarantees that all the calls contained within the transaction will be carried out in order, without any interference from other, concurrent transactions. If two or more transactions start up at the same time, the system ensures that the final result is the same as if they were all run in some (undefined) sequential order.

The classical example of where transactions make programming much

easier is in a banking system. Imagine that a certain bank account contains 100 dollars, and that two processes are each trying to add 50 dollars to it. In an unconstrained system, each process might simultaneously read the file containing the current balance (100), individually compute the new balance (150), and successively overwrite the file with this new value. The final result could either be 150 or 200, depending on the precise timing of the reading and writing. By grouping all the operations into a transaction, interleaving cannot occur and the final result will always be 200.

In Fig. 5-5 we summarize the four approaches we have discussed for dealing with shared files in a distributed system.

Method	Comment
UNIX semantics	Every operation on a file is instantly visible to all processes
Session semantics	No changes are visible to other processes until the file is closed
Immutable files	No updates are possible; simplifies sharing and replication
Transactions	All changes have the all-or-nothing property

Fig. 5-5. Four ways of dealing with the shared files in a distributed system.

5.2. DISTRIBUTED FILE SYSTEM IMPLEMENTATION

In the preceding section, we have described various aspects of distributed file systems from the user's perspective, that is, how they appear to the user. In this section we will see how these systems are implemented. We will start out by presenting some experimental information about file usage. Then we will go on to look at system structure, the implementation of caching, replication in distributed systems, and concurrency control. We will conclude with a short discussion of some lessons that have been learned from experience.

5.2.1. File Usage

Before implementing any system, distributed or otherwise, it is useful to have a good idea of how it will be used, to make sure that the most commonly executed operations will be efficient. To this end, Satyanarayanan (1981) made a study of file usage patterns. We will present his major results below.

However, first, a few words of warning about these and similar measurements are in order. Some of the measurements are static, meaning that they represent a snapshot of the system at a certain instant. Static measurements are made by examining the disk to see what is on it. These measurements include the distribution of file sizes, the distribution of file types, and the amount of storage occupied by files of various types and sizes. Other measurements are dynamic, made by modifying the file system to record all operations to a log for subsequent analysis. These data yield information about the relative frequency of various operations, the number of files open at any moment, and the amount of sharing that takes place. By combining the static and dynamic measurements, even though they are fundamentally different, we can get a better picture of how the file system is used.

One problem that always occurs with measurements of any existing system is knowing how typical the observed user population is. Satyanarayanan's measurements were made at a university. Do they also apply to industrial research labs? To office automation projects? To banking systems? No one really knows for sure until these systems, too, are instrumented and measured.

Another problem inherent in making measurements is watching out for artifacts of the system being measured. As a simple example, when looking at the distribution of file names in an MS-DOS system, one could quickly conclude that file names are never more than eight characters (plus an optional three-character extension). However, it would be a mistake to draw the conclusion that eight characters are therefore enough, since nobody ever uses more than eight characters. Since MS-DOS does not allow more than eight characters in a file name, it is impossible to tell what users would do if they were not constrained to eight-character file names.

Finally, Satyanarayanan's measurements were made on more-or-less traditional UNIX systems. Whether or not they can be transferred or extrapolated to distributed systems is not really known.

This being said, the most important conclusions are listed in Fig. 5-6. From these observations, one can draw certain conclusions. To start with, most files are under 10K, which agrees with the results of Mullender and Tanenbaum (1984) made under different circumstances. This observation suggests that it may be feasible to transfer entire files rather than disk blocks between server and client. Since whole file transfer is typically simpler and more efficient, this idea is worth considering. Of course, some files are large, so provision has to be made for them too. Still, a good guideline is to optimize for the normal case and treat the abnormal case specially.

An interesting observation is that most files have short lifetimes. A common pattern is to create a file, read it (probably once), and then delete it. A typical usage might be a compiler that creates temporary files for transmitting information between its passes. The implication here is that it is probably a good

Most files are small (less than 10 K)
Reading is much more common than writing
Reads and writes are sequential; random access is rare
Most files have a short lifetime
File sharing is unusual
The average process uses only a few files
Distinct file classes with different properties exist

Fig. 5-6. Observed file system properties.

idea to create the file on the client and keep it there until it is deleted. Doing so may eliminate a considerable amount of unnecessary client-server traffic.

The fact that few files are shared argues for client caching. As we have seen already, caching makes the semantics more complicated, but if files are rarely shared, it may well be best to do client caching and accept the consequences of session semantics in return for the better performance.

Finally, the clear existence of distinct file classes suggests that perhaps different mechanisms should be used to handle the different classes. System binaries need to be widespread but hardly ever change, so they should probably be widely replicated, even if this means that an occasional update is complex. Compiler and other temporary files are short, unshared, and disappear quickly, so they should be kept locally wherever possible. Electronic mailboxes are frequently updated but rarely shared, so replication is not likely to gain anything. Ordinary data files may be shared, so they may need still other handling.

5.2.2. System Structure

In this section we will look at some of the ways that file servers and directory servers are organized internally, with special attention to alternative approaches. Let us start with a very simple question: Are clients and servers different? Surprisingly enough, there is no agreement on this matter.

In some systems, there is no distinction between clients and servers. All machines run the same basic software, so any machine wanting to offer file service to the public is free to do so. Offering file service is just a matter of exporting the names of selected directories so that other machines can access them.

In other systems, the file server and directory server are just user programs, so a system can be configured to run client and server software on the same

machines or not, as it wishes. Finally, at the other extreme, are systems in which clients and servers are fundamentally different machines, in terms of either hardware or software. The servers may even run a different version of the operating system from the clients. While separation of function may seem a bit cleaner, there is no fundamental reason to prefer one approach over the others.

A second implementation issue on which systems differ is how the file and directory service is structured. One organization is to combine the two into a single server that handles all the directory and file calls itself. Another possibility, however, is to keep them separate. In the latter case, opening a file requires going to the directory server to map its symbolic name onto its binary name (e.g., machine + i-node) and then going to the file server with the binary name to read or write the file.

Arguing in favor of the split is that the two functions are really unrelated, so keeping them separate is more flexible. For example, one could implement an MS-DOS directory server and a UNIX directory server, both of which use the same file server for physical storage. Separation of function is also likely to produce simpler software. Weighing against this is that having two servers requires more communication.

Let us consider the case of separate directory and file servers for the moment. In the normal case, the client sends a symbolic name to the directory server, which then returns the binary name that the file server understands. However, it is possible for a directory hierarchy to be partitioned among multiple servers, as illustrated in Fig. 5-7. Suppose, for example, that we have a system in which the current directory, on server 1, contains an entry, *a*, for another directory on server 2. Similarly, this directory contains an entry, *b*, for a directory on server 3. This third directory contains an entry for a file *c*, along with its binary name.

To look up *a*/*b*/*c*, the client sends a message to server 1, which manages its current directory. The server finds *a*, but sees that the binary name refers to another server. It now has a choice. It can either tell the client which server holds *b* and have the client look up *b*/*c* there itself, as shown in Fig. 5-7(a), or it can forward the remainder of the request to server 2 itself and not reply at all, as shown in Fig. 5-7(b). The former scheme requires the client to be aware of which server holds which directory, and requires more messages. The latter method is more efficient, but cannot be handled using normal RPC since the process to which the client sends the message is not the one that sends the reply.

Looking up path names all the time, especially if multiple directory servers are involved, can be expensive. Some systems attempt to improve their performance by maintaining a cache of hints, that is, recently looked up names and the results of these lookups. When a file is opened, the cache is checked to see if the path name is there. If so, the directory-by-directory lookup is skipped and the binary address is taken from the cache. If not, it is looked up.

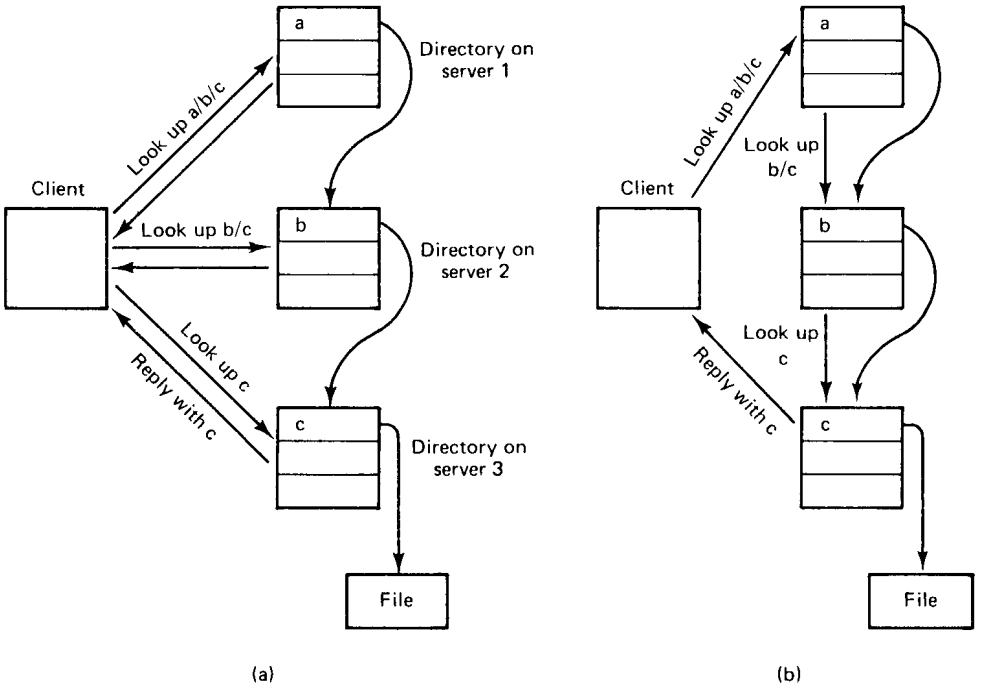


Fig. 5-7. (a) Iterative lookup of *a/b/c*. (b) Automatic lookup.

For name caching to work, it is essential that when an obsolete binary name is used inadvertently, the client is somehow informed so it can fall back on the directory-by-directory lookup to find the file and update the cache. Furthermore, to make hint caching worthwhile in the first place, the hints have to be right most of the time. When these conditions are fulfilled, caching hints can be a powerful technique that is applicable in many distributed operating systems.

The final structural issue that we will consider here is whether or not file, directory, and other servers should maintain state information about clients. This issue is moderately controversial, with two competing schools of thought in existence.

One school thinks that servers should be **stateless**. In other words, when a client sends a request to a server, the server carries out the request, sends the reply, and then removes from its internal tables all information about the request. Between requests, no client-specific information is kept on the server. The other school of thought maintains that it is all right for servers to maintain state information about clients between requests. After all, centralized operating systems maintain state information about active processes, so why should this traditional behavior suddenly become unacceptable?

To better understand the difference, consider a file server that has commands to open, read, write, and close files. After a file has been opened, the server must maintain information about which client has which file open. Typically, when a file is opened, the client is given a file descriptor or other number which is used in subsequent calls to identify the file. When a request comes in, the server uses the file descriptor to determine which file is needed. The table mapping the file descriptors onto the files themselves is state information.

With a stateless server, each request must be self-contained. It must contain the full file name and the offset within the file, in order to allow the server to do the work. This information increases message length.

Another way to look at state information is to consider what happens if a server crashes and all its tables are lost forever. When the server is rebooted, it no longer knows which clients have which files open. Subsequent attempts to read and write open files will then fail, and recovery, if possible at all, will be entirely up to the clients. As a consequence, stateless servers tend to be more fault tolerant than those that maintain state, which is one of the arguments in favor of the former.

Advantages of stateless servers	Advantages of stateful servers
Fault tolerance	Shorter request messages
No OPEN/CLOSE calls needed	Better performance
No server space wasted on tables	Readahead possible
No limits on number of open files	Idempotency easier
No problems if a client crashes	File locking possible

Fig. 5-8. A comparison of stateless and stateful servers.

The arguments both ways are summarized in Fig. 5-8. Stateless servers are inherently more fault tolerant, as we just mentioned. OPEN and CLOSE calls are not needed, which reduces the number of messages, especially for the common case in which the entire file is read in a single blow. No server space is wasted on tables. When tables are used, if too many clients have too many files open at once, the tables can fill up and new files cannot be opened. Finally, with a stateful server, if a client crashes when a file is open, the server is in a bind. If it does nothing, its tables will eventually fill up with junk. If it times out inactive open files, a client that happens to wait too long between requests will be refused service, and correct programs will fail to function correctly. Statelessness eliminates these problems.

Stateful servers also have things going for them. Since READ and WRITE

messages do not have to contain file names, they can be shorter, thus using less network bandwidth. Better performance is frequently possible since information about open files (in UNIX terms, the i-nodes) can be kept in main memory until the files are closed. Blocks can be read in advance to reduce delay, since most files are read sequentially. If a client ever times out and sends the same request twice, for example, APPEND, it is much easier to detect this with state (by having a sequence number in each message). Achieving idempotency in the face of unreliable communication with stateless operation takes more thought and effort. Finally, file locking is impossible to do in a truly stateless system, since the only effect setting a lock has is to enter state into the system. In stateless systems, file locking has to be done by a special lock server.

5.2.3. Caching

In a client-server system, each with main memory and a disk, there are four potential places to store files, or parts of files: the server's disk, the server's main memory, the client's disk (if available), or the client's main memory, as illustrated in Fig. 5-9. These different storage locations all have different properties, as we shall see.

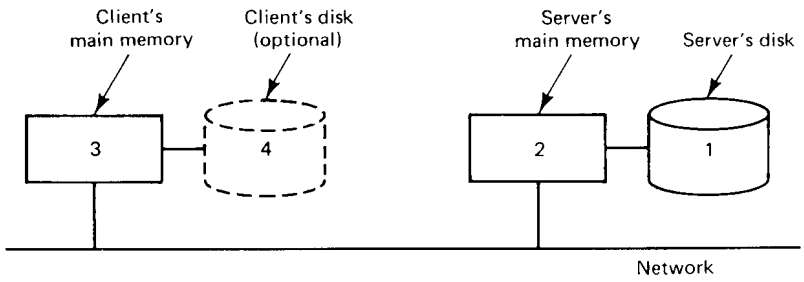


Fig. 5-9. Four places to store files or parts of files.

The most straightforward place to store all files is on the server's disk. There is generally plenty of space there and the files are then accessible to all clients. Furthermore, with only one copy of each file, no consistency problems arise.

The problem with using the server's disk is performance. Before a client can read a file, the file must be transferred from the server's disk to the server's main memory, and then again over the network to the client's main memory. Both transfers take time.

A considerable performance gain can be achieved by **caching** (i.e., holding) the most recently used files in the server's main memory. A client reading a file that happens to be in the server's cache eliminates the disk transfer, although the

network transfer still has to be done. Since main memory is invariably smaller than the disk, some algorithm is needed to determine which files or parts of files should be kept in the cache.

This algorithm has two problems to solve. First, what is the unit the cache manages? It can be either whole files or disk blocks. If entire files are cached, they can be stored contiguously on the disk (or at least in very large chunks), allowing high-speed transfers between memory and disk and generally good performance. Disk block caching, however, uses cache and disk space more efficiently.

Second, the algorithm must decide what to do when the cache fills up and something must be evicted. Any of the standard caching algorithms can be used here, but because cache references are so infrequent compared to memory references, an exact implementation of LRU using linked lists is generally feasible. When something has to be evicted, the oldest one is chosen. If an up-to-date copy exists on disk, the cache copy is just discarded. Otherwise, the disk is first updated.

Having a cache in the server's main memory is easy to do and totally transparent to the clients. Since the server can keep its memory and disk copies synchronized, from the clients' point of view, there is only one copy of each file, so no consistency problems arise.

Although server caching eliminates a disk transfer on each access, it still has a network access. The only way to get rid of the network access is to do caching on the client side, which is where all the problems come in. The trade-off between using the client's main memory or its disk is one of space versus performance. The disk holds more but is slower. When faced with a choice between having a cache in the server's main memory versus the client's disk, the former is usually somewhat faster, and it is always much simpler. Of course, if large amounts of data are being used, a client disk cache may be better. In any event, most systems that do client caching do it in the client's main memory, so we will concentrate on that.

If the designers decide to put the cache in the client's main memory, three options are open as to precisely where to put it. The simplest is to cache files directly inside each user process' own address space, as shown in Fig. 5-10(b). Typically, the cache is managed by the system call library. As files are opened, closed, read, and written, the library simply keeps the most heavily used ones around, so that when a file is reused, it may already be available. When the process exits, all modified files are written back to the server. Although this scheme has an extremely low overhead, it is effective only if individual processes open and close files repeatedly. A data base manager process might fit this description, but in the usual program development environment, most processes read each file only once, so caching within the library wins nothing.

The second place to put the cache is in the kernel, as shown in Fig. 5-10(c).

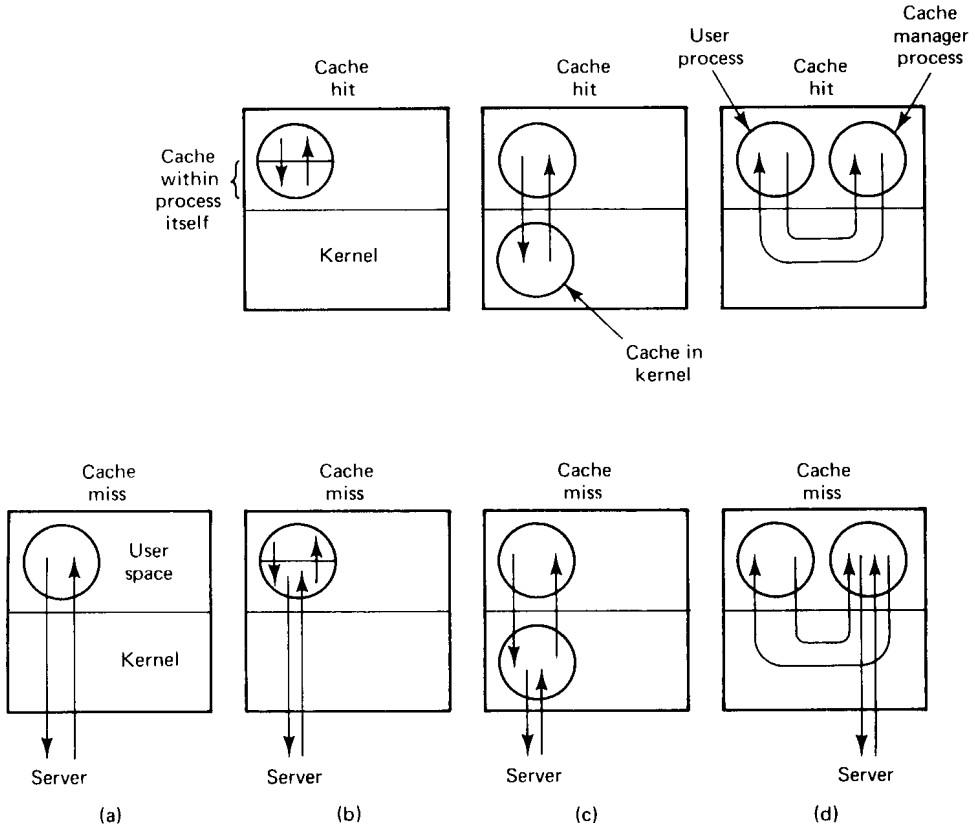


Fig. 5-10. Various ways of doing caching in client memory. (a) No caching. (b) Caching within each process. (c) Caching in the kernel. (d) The cache manager as a user process.

The disadvantage here is that a kernel call is needed in all cases, even on a cache hit, but the fact that the cache survives the process more than compensates. For example, suppose that a two-pass compiler runs as two processes. Pass one writes an intermediate file read by pass two. In Fig. 5-10(c), after the pass one process terminates, the intermediate file will probably be in the cache, so no server calls will have to be made when the pass two process reads it in.

The third place for the cache is in a separate user-level cache manager process, as shown in Fig. 5-10(d). The advantage of a user-level cache manager is that it keeps the (micro)kernel free of file system code, is easier to program because it is completely isolated, and is more flexible.

On the other hand, when the kernel manages the cache, it can dynamically decide how much memory to reserve for programs and how much for the cache.

With a user-level cache manager running on a machine with virtual memory, it is conceivable that the kernel could decide to page out some or all of the cache to a disk, so that a so-called "cache hit" requires one or more pages to be brought in. Needless to say, this defeats the idea of client caching completely. However, if it is possible for the cache manager to allocate and lock in memory some number of pages, this ironic situation can be avoided.

When evaluating whether caching is worth the trouble at all, it is important to note that in Fig. 5-10(a), it takes exactly one RPC to make a file request, no matter what. In both Fig. 5-10(c) and Fig. 5-10(d) it takes either one or two, depending on whether or not the request can be satisfied out of the cache. Thus the mean number of RPCs is always greater when caching is used. In a situation in which RPCs are fast and network transfers are slow (fast CPUs, slow networks), caching can give a big gain in performance. If, however, network transfers are very fast (e.g., with high-speed fiber optic networks), the network transfer time will matter less, so the extra RPCs may eat up a substantial fraction of the gain. Thus the performance gain provided by caching depends to some extent on the CPU and network technology available, and of course, on the applications.

Cache Consistency

As usual in computer science, you never get something for nothing. Client caching introduces inconsistency into the system. If two clients simultaneously read the same file and then both modify it, several problems occur. For one, when a third process reads the file from the server, it will get the original version, not one of the two new ones. This problem can be defined away by adopting session semantics (officially stating that the effects of modifying a file are not supposed to be visible globally until the file is closed). In other words, this "incorrect" behavior is simply declared to be the "correct" behavior. Of course, if the user expects UNIX semantics, the trick does not work.

Another problem, unfortunately, that cannot be defined away at all is that when the two files are written back to the server, the one written last will overwrite the other one. The moral of the story is that client caching has to be thought out fairly carefully. Below we will discuss some of the problems and proposed solutions.

One way to solve the consistency problem is to use the **write-through algorithm.** When a cache entry (file or block) is modified, the new value is kept in the cache, but is also sent immediately to the server. As a consequence, when another process reads the file, it gets the most recent value.

However, the following problem arises. Suppose that a client process on machine *A* reads a file, *f*. The client terminates but the machine keeps *f* in its cache. Later, a client on machine *B* reads the same file, modifies it, and writes it

through to the server. Finally, a new client process is started up on machine *A*. The first thing it does is open and read *f*, which is taken from the cache. Unfortunately, the value there is now obsolete.

A possible way out is to require the cache manager to check with the server before providing any client with a file from the cache. This check could be done by comparing the time of last modification of the cached version with the server's version. If they are the same, the cache is up-to-date. If not, the current version must be fetched from the server. Instead of using dates, version numbers or checksums can also be used. Although going to the server to verify dates, version numbers, or checksums takes an RPC, the amount of data exchanged is small. Still, it takes some time.

Another trouble with the write-through algorithm is that although it helps on reads, the network traffic for writes is the same as if there were no caching at all. Many system designers find this unacceptable, and cheat: instead of going to the server the instant the write is done, the client just makes a note that a file has been updated. Once every 30 seconds or so, all the file updates are gathered together and sent to the server all at once. A single bulk write is usually more efficient than many small ones.

Besides, many programs create scratch files, write them, read them back, and then delete them, all in quick succession. In the event that this entire sequence happens before it is time to send all modified files back to the server, the now-deleted file does not have to be written back at all. Not having to use the file server at all for temporary files can be a major performance gain.

Of course, delaying the writes muddies the semantics, because when another process reads the file, what it gets depends on the timing. Thus postponing the writes is a trade-off between better performance and cleaner semantics (which translates into easier programming).

The next step in this direction is to adopt session semantics and write a file back to the server only after it has been closed. This algorithm is called **write-on-close**. Better yet, wait 30 seconds after the close to see if the file is going to be deleted. As we saw earlier, going this route means that if two cached files are written back in succession, the second one overwrites the first one. The only solution to this problem is to note that it is not nearly as bad as it first appears. In a single CPU system, it is possible for two processes to open and read a file, modify it within their respective address spaces, and then write it back. Consequently, write-on-close with session semantics is not that much worse than what can happen on a single CPU system.

A completely different approach to consistency is to use a centralized control algorithm. When a file is opened, the machine opening it sends a message to the file server to announce this fact. The file server keeps track of who has which file open, and whether it is open for reading, writing, or both. If a file is open for reading, there is no problem with letting other processes open it for

reading, but opening it for writing must be avoided. Similarly, if some process has a file open for writing, all other accesses must be prevented. When a file is closed, this event must be reported, so the server can update its tables telling which client has which file open. The modified file can also be shipped back to the server at this point.

When a client tries to open a file and the file is already open elsewhere in the system, the new request can either be denied or queued. Alternatively, the server can send an **unsolicited message** to all clients having the file open, telling them to remove that file from their caches and disable caching just for that one file. In this way, multiple readers and writers can run simultaneously, with the results being no better and no worse than would be achieved on a single CPU system.

Although sending unsolicited messages is clearly possible, it is inelegant, since it reverses the client and server roles. Normally, servers do not spontaneously send messages to clients or initiate RPCs with them. If the clients are multithreaded, one thread can be permanently allocated to waiting for server requests, but if they are not, the unsolicited message must cause an interrupt.

Even with these precautions, one must be careful. In particular, if a machine opens, caches, and then closes a file, upon opening it again the cache manager must still check to see if the cache is valid. After all, some other process might have subsequently opened, modified, and closed the file. Many variations of this centralized control algorithm are possible, with differing semantics. For example, servers can keep track of cached files, rather than open files. All these methods have a single point of failure and none of them scale well to large systems.

Method	Comments
Write through	Works, but does not affect write traffic
Delayed write	Better performance but possibly ambiguous semantics
Write on close	Matches session semantics
Centralized control	UNIX semantics, but not robust and scales poorly

Fig. 5-11. Four algorithms for managing a client file cache.

The four cache management algorithms discussed above are summarized in Fig. 5-11. To summarize the subject of caching as a whole, server caching is easy to do and almost always worth the trouble, independent of whether client caching is present or not. Server caching has no effect on the file system semantics seen by the clients. Client caching, in contrast, offers better performance at the price of increased complexity and possibly fuzzier semantics. Whether it is

worth doing or not depends on how the designers feel about performance, complexity, and ease of programming.

Earlier in this chapter, when we were discussing the semantics of distributed file systems, we pointed out that one of the design options is immutable files. One of the great attractions of an immutable file is the ability to cache it on machine *A* without having to worry about the possibility that machine *B* will change it. Changes are not permitted. Of course, a new file may have been created and bound to the same symbolic name as the cached file, but this can be checked for whenever a cached file is reopened. This model has the same RPC overhead discussed above, but the semantics are less fuzzy.

5.2.4. Replication

Distributed file systems often provide file replication as a service to their clients. In other words, multiple copies of selected files are maintained, with each copy on a separate file server. The reasons for offering such a service vary, but among the major reasons are:

1. To increase reliability by having independent backups of each file. If one server goes down, or is even lost permanently, no data are lost. For many applications, this property is extremely desirable.
2. To allow file access to occur even if one file server is down. The motto here is: The show must go on. A server crash should not bring the entire system down until the server can be rebooted.
3. To split the workload over multiple servers. As the system grows in size, having all the files on one server can become a performance bottleneck. By having files replicated on two or more servers, the least heavily loaded one can be used.

The first two relate to improving reliability and availability; the third concerns performance. All are important.

A key issue relating to replication is transparency (as usual). To what extent are the users aware that some files are replicated? Do they play any role in the replication process, or is it handled entirely automatically? At one extreme, the users are fully aware of the replication process and can even control it. At the other, the system does everything behind their backs. In the latter case, we say that the system is **replication transparent**.

Figure 5-12 shows three ways replication can be done. The first way, shown in Fig. 5-12(a), is for the programmer to control the entire process. When a process makes a file, it does so on one specific server. Then it can make additional copies on other servers, if desired. If the directory server permits multiple

copies of a file, the network addresses of all copies can then be associated with the file name, as shown at the bottom of Fig. 5-12(a), so that when the name is looked up, all copies will be found. When the file is subsequently opened, the copies can be tried sequentially in some order, until an available one is found.

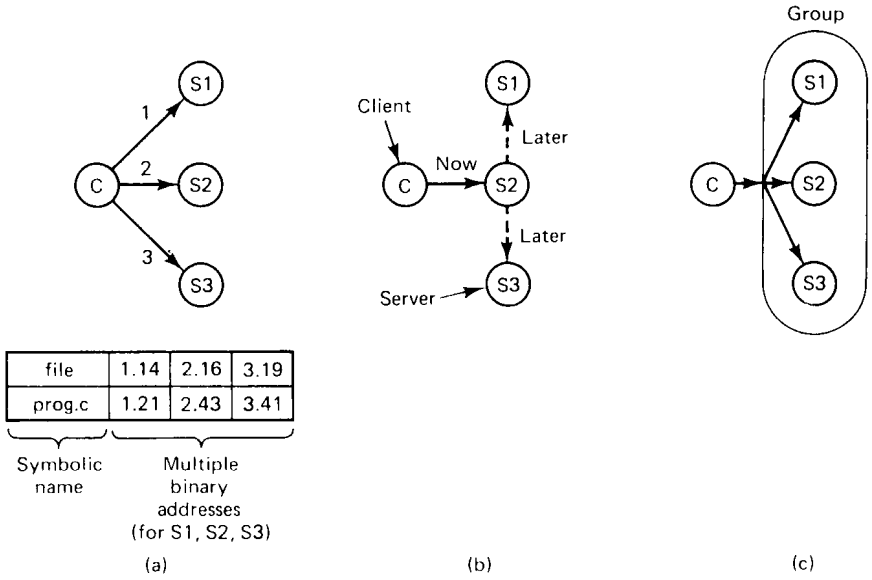


Fig. 5-12. (a) Explicit file replication. (b) Lazy file replication. (c) File replication using a group.

To make the concept of explicit replication more familiar, consider how it can be done in a system based on remote mounting in UNIX. Suppose that a programmer's home directory is */machine1/usr/ast*. After creating a file, for example the file, */machine1/usr/ast/xyz*, the programmer, process, or library can use the *cp* command (or equivalent) to make copies in */machine2/usr/ast/xyz* and */machine3/usr/ast/xyz*. Programs can be written to accept strings like */usr/ast/xyz* as arguments, and successively try to open the copies until one succeeds. While this scheme can be made to work, it is a lot of trouble. For this reason, a distributed system should do better.

In Fig. 5-12(b) we see an alternative approach, **lazy replication**. Here, only one copy of each file is created, on some server. Later, the server itself makes replicas on other servers automatically, without the programmer's knowledge. The system must be smart enough to be able to retrieve any of these copies if need be. When making copies in the background like this, it is important to pay attention to the possibility that the file might change before the copies can be made.

Our final method is to use group communication, as shown in Fig. 5-13(c).

In this scheme, all WRITE system calls are simultaneously transmitted to all the servers, so extra copies are made at the same time the original is made. There are two principal differences between lazy replication and using a group. First, with lazy replication, one server is addressed rather than a group. Second, lazy replication happens in the background, when the server has some free time, whereas when group communication is used, all copies are made at the same time.

Update Protocols

Above we looked at the problem of how replicated files can be created. Now let us see how existing ones can be modified. Just sending an update message to each copy in sequence is not a good idea because if the process doing the update crashes partway through, some copies will be changed and others not. As a result, some future reads may get the old value and others may get the new value, hardly a desirable situation. We will now look at two well-known algorithms that solve this problem.

The first algorithm is called **primary copy replication**. When it is used, one server is designated as the primary. All the others are secondaries. When a replicated file is to be updated, the change is sent to the primary server, which makes the change locally and then sends commands to the secondaries, ordering them to change, too. Reads can be done from any copy, primary or secondary.

To guard against the situation that the primary crashes before it has had a chance to instruct all the secondaries, the update should be written to stable storage prior to changing the primary copy. In this way, when a server reboots after a crash, a check can be made to see if any updates were in progress at the time of the crash. If so, they can still be carried out. Sooner or later, all the secondaries will be updated.

Although the method is straightforward, it has the disadvantage that if the primary is down, no updates can be performed. To get around this asymmetry, Gifford (1979) proposed a more robust method, known as **voting**. The basic idea is to require clients to request and acquire the permission of multiple servers before either reading or writing a replicated file.

As a simple example of how the algorithm works, suppose that a file is replicated on N servers. We could make a rule stating that to update a file, a client must first contact at least half the servers plus 1 (a majority) and get them to agree to do the update. Once they have agreed, the file is changed and a new version number is associated with the new file. The version number is used to identify the version of the file and is the same for all the newly updated files.

To read a replicated file, a client must also contact at least half the servers plus 1 and ask them to send the version numbers associated with the file. If all the version numbers agree, this must be the most recent version because an

attempt to update only the remaining servers would fail because there are not enough of them.

For example, if there are five servers and a client determines that three of them have version 8, it is impossible that the other two have version 9. After all, any successful update from version 8 to version 9 requires getting three servers to agree to it, not just two.

Gifford's scheme is actually somewhat more general than this. In it, to read a file of which N replicas exist, a client needs to assemble a **read quorum**, an arbitrary collection of any N_r servers, or more. Similarly, to modify a file, a **write quorum** of at least N_w servers is required. The values of N_r and N_w are subject to the constraint that $N_r + N_w > N$. Only after the appropriate number of servers has agreed to participate can a file be read or written.

To see how this algorithm works, consider Fig. 5-13(a), which has $N_r = 3$ and $N_w = 10$. Imagine that the most recent write quorum consisted of the 10 servers C through L. All of these get the new version and the new version number. Any subsequent read quorum of three servers will have to contain at least one member of this set. When the client looks at the version numbers, it will know which is most recent and take that one.

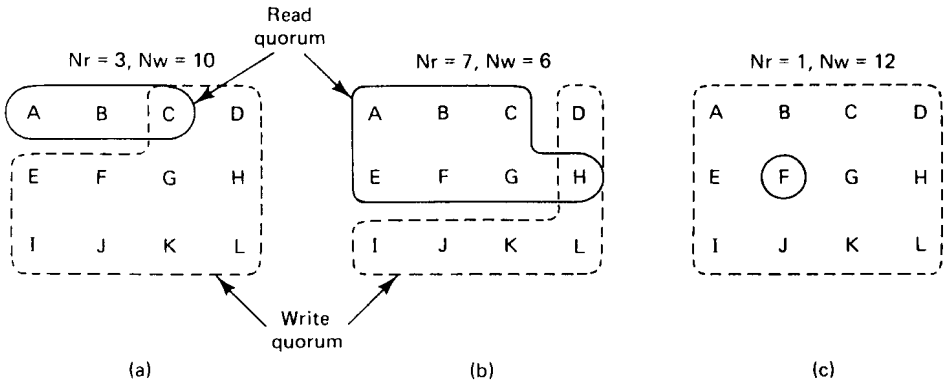


Fig. 5-13. Three examples of the voting algorithm.

In Fig. 5-13(b) and (c), we see two more examples. The latter is especially interesting because it sets N_r to 1, making it possible to read a replicated file by finding any copy and using it. The price paid, however, is that write updates need to acquire all copies.

An interesting variation on voting is **voting with ghosts** (Van Renesse and Tanenbaum, 1988). In most applications, reads are much more common than writes, so N_r is typically a small number and N_w is nearly N. This choice means that if a few servers are down, it may be impossible to obtain a write quorum.

Voting with ghosts solves this problem by creating a dummy server, with no

storage, for each real server that is down. A ghost is not permitted in a read quorum (it does not have any files, after all), but it may join a write quorum, in which case it just throws away the file written to it. A write succeeds only if at least one server is real.

When a failed server is rebooted, it must obtain a read quorum to locate the most recent version, which it then copies to itself before starting normal operation. The algorithm works because it has the same property as the basic voting scheme, namely, N_r and N_w are chosen so that acquiring a read quorum and a write quorum at the same time is impossible. The only difference here is that dead machines are allowed in a write quorum, subject to the condition that when they come back up they immediately obtain the current version before going into service.

Other replication algorithms are described in (Bernstein and Goodman, 1984; Brereton, 1986; Pu et al., 1986; and Purdin et al., 1987).

5.2.5. An Example: Sun's Network File System

In this section we will examine an example network file system, Sun Microsystem's **Network File System**, universally known as **NFS**. NFS was originally designed and implemented by Sun Microsystems for use on its UNIX-based workstations. Other manufacturers now support it as well, for both UNIX and other operating systems (including MS-DOS). NFS supports heterogeneous systems, for example, MS-DOS clients making use of UNIX servers. It is not even required that all the machines use the same hardware. It is common to find MS-DOS clients running on Intel 386 CPUs getting service from UNIX file servers running on Motorola 68030 or Sun SPARC CPUs.

Three aspects of NFS are of interest: the architecture, the protocol, and the implementation. Let us look at these in turn.

NFS Architecture

The basic idea behind NFS is to allow an arbitrary collection of clients and servers to share a common file system. In most cases, all the clients and servers are on the same LAN, but this is not required. It is possible to run NFS over a wide-area network. For simplicity we will speak of clients and servers as though they were on distinct machines, but in fact, NFS allows every machine to be both a client and a server at the same time.

Each NFS server exports one or more of its directories for access by remote clients. When a directory is made available, so are all of its subdirectories, so in fact, entire directory trees are normally exported as a unit. The list of directories a server exports is maintained in the */etc/exports* file, so these directories can be exported automatically whenever the server is booted.

Clients access exported directories by mounting them. When a client mounts a (remote) directory, it becomes part of its directory hierarchy, as shown in Fig. 5-13. Many Sun workstations are diskless. If it so desires, a diskless client can mount a remote file system on its root directory, resulting in a file system that is supported entirely on a remote server. Those workstations that do have local disks can mount remote directories anywhere they wish on top of their local directory hierarchy, resulting in a file system that is partly local and partly remote. To programs running on the client machine, there is (almost) no difference between a file located on a remote file server and a file located on the local disk.

Thus the basic architectural characteristic of NFS is that servers export directories and clients mount them remotely. If two or more clients mount the same directory at the same time, they can communicate by sharing files in their common directories. A program on one client can create a file, and a program on a different one can read the file. Once the mounts have been done, nothing special has to be done to achieve sharing. The shared files are just there in the directory hierarchy of multiple machines and can be read and written the usual way. This simplicity is one of the great attractions of NFS.

NFS Protocols

Since one of the goals of NFS is to support a heterogeneous system, with clients and servers possibly running different operating systems on different hardware, it is essential that the interface between the clients and servers be well defined. Only then is it possible for anyone to be able to write a new client implementation and expect it to work correctly with existing servers, and vice versa.

NFS accomplishes this goal by defining two client-server protocols. A **protocol** is a set of requests sent by clients to servers, along with the corresponding replies sent by the servers back to the clients. (Protocols are an important topic in distributed systems; we will come back to them later in more detail.) As long as a server recognizes and can handle all the requests in the protocols, it need not know anything at all about its clients. Similarly, clients can treat servers as "black boxes" that accept and process a specific set of requests. How they do it is their own business.

The first NFS protocol handles mounting. A client can send a path name to a server and request permission to mount that directory somewhere in its directory hierarchy. The place where it is to be mounted is not contained in the message, as the server does not care where it is to be mounted. If the path name is legal and the directory specified has been exported, the server returns a **file handle** to the client. The file handle contains fields uniquely identifying the file system type, the disk, the i-node number of the directory, and security

information. Subsequent calls to read and write files in the mounted directory use the file handle.

Many clients are configured to mount certain remote directories without manual intervention. Typically, these clients contain a file called */etc/rc*, which is a shell script containing the remote mount commands. This shell script is executed automatically when the client is booted.

Alternatively, Sun's version of UNIX also supports **automounting**. This feature allows a set of remote directories to be associated with a local directory. None of these remote directories are mounted (or their servers even contacted) when the client is booted. Instead, the first time a remote file is opened, the operating system sends a message to each of the servers. The first one to reply wins, and its directory is mounted.

Automounting has two principal advantages over static mounting via the */etc/rc* file. First, if one of the NFS servers named in */etc/rc* happens to be down, it is impossible to bring the client up, at least not without some difficulty, delay, and quite a few error messages. If the user does not even need that server at the moment, all that work is wasted. Second, by allowing the client to try a set of servers in parallel, a degree of fault tolerance can be achieved (because only one of them need to be up), and the performance can be improved (by choosing the first one to reply—presumably the least heavily loaded).

On the other hand, it is tacitly assumed that all the file systems specified as alternatives for the automount are identical. Since NFS provides no support for file or directory replication, it is up to the user to arrange for all the file systems to be the same. Consequently, automounting is most often used for read-only file systems containing system binaries and other files that rarely change.

The second NFS protocol is for directory and file access. Clients can send messages to servers to manipulate directories and to read and write files. In addition, they can also access file attributes, such as file mode, size, and time of last modification. Most UNIX system calls are supported by NFS, with the perhaps surprising exception of OPEN and CLOSE.

The omission of OPEN and CLOSE is not an accident. It is fully intentional. It is not necessary to open a file before reading it, nor to close it when done. Instead, to read a file, a client sends the server a message containing the file name, with a request to look it up and return a file handle, which is a structure that identifies the file. Unlike an OPEN call, this LOOKUP operation does not copy any information into internal system tables. The READ call contains the file handle of the file to read, the offset in the file to begin reading, and the number of bytes desired. Each such message is self-contained. The advantage of this scheme is that the server does not have to remember anything about open connections in between calls to it. Thus if a server crashes and then recovers, no information about open files is lost, because there is none. A server like this that does not maintain state information about open files is said to be **stateless**.

In contrast, in UNIX System V, the **Remote File System (RFS)** requires a file to be opened before it can be read or written. The server then makes a table entry keeping track of the fact that the file is open, and where the reader currently is, so each request need not carry an offset. The disadvantage of this scheme is that if a server crashes and then quickly reboots, all open connections are lost, and client programs fail. NFS does not have this property.

Unfortunately, the NFS method makes it difficult to achieve the exact UNIX file semantics. For example, in UNIX a file can be opened and locked so that other processes cannot access it. When the file is closed, the locks are released. In a stateless server such as NFS, locks cannot be associated with open files, because the server does not know which files are open. NFS therefore needs a separate, additional mechanism to handle locking.

NFS uses the UNIX protection mechanism, with the *rwx* bits for the owner, group, and others. Originally, each request message simply contained the user and group ids of the caller, which the NFS server used to validate the access. In effect, it trusted the clients not to cheat. Several years' experience abundantly demonstrated that such an assumption was—how shall we put it?—naive. Currently, public key cryptography can be used to establish a secure key for validating the client and server on each request and reply. When this option is enabled, a malicious client cannot impersonate another client because it does not know that client's secret key. As an aside, cryptography is used only to authenticate the parties. The data themselves are never encrypted.

All the keys used for the authentication, as well as other information are maintained by the **NIS (Network Information Service)**. The NIS was formerly known as the **yellow pages**. Its function is to store (key, value) pairs. When a key is provided, it returns the corresponding value. Not only does it handle encryption keys, but it also stores the mapping of user names to (encrypted) passwords, as well as the mapping of machine names to network addresses, and other items.

The network information servers are replicated using a master/slave arrangement. To read their data, a process can use either the master or any of the copies (slaves). However, all changes must be made only to the master, which then propagates them to the slaves. There is a short interval after an update in which the data base is inconsistent.

NFS Implementation

Although the implementation of the client and server code is independent of the NFS protocols, it is interesting to take a quick peek at Sun's implementation. It consists of three layers, as shown in Fig. 5-14. The top layer is the system call layer. This handles calls like OPEN, READ, and CLOSE. After parsing the call

and checking the parameters, it invokes the second layer, the virtual file system (VFS) layer.

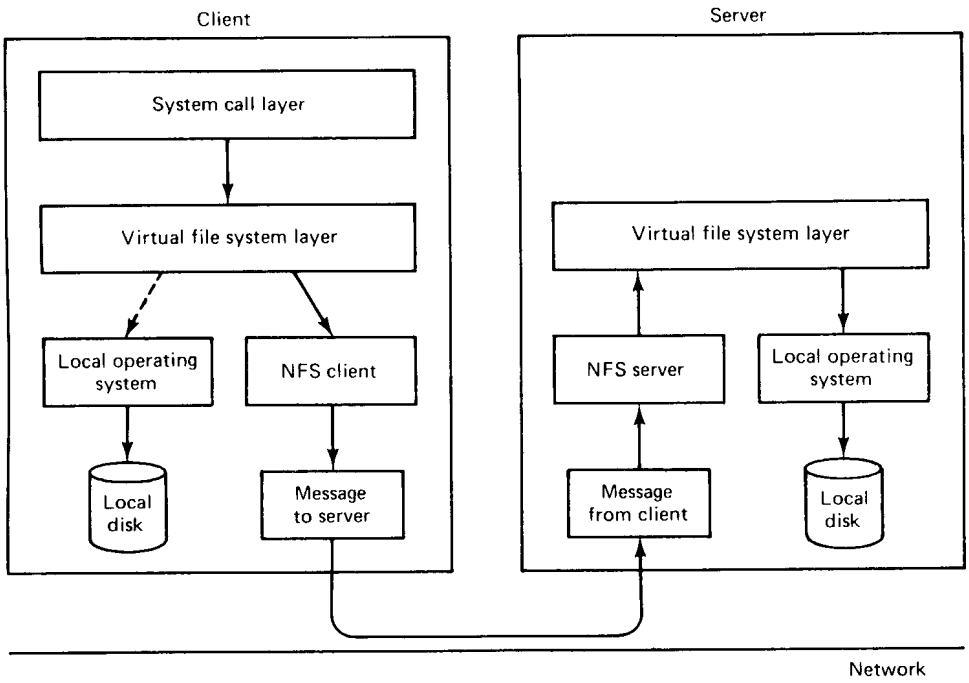


Fig. 5-14. NFS layer structure.

The task of the VFS layer is to maintain a table with one entry for each open file, analogous to the table of i-nodes for open files in UNIX. In ordinary UNIX, an i-node is indicated uniquely by a (device, i-node number) pair. Instead, the VFS layer has an entry, called a **v-node** (**virtual i-node**), for every open file. V-nodes are used to tell whether the file is local or remote. For remote files, enough information is provided to be able to access them.

To see how v-nodes are used, let us trace a sequence of MOUNT, OPEN, and READ system calls. To mount a remote file system, the system administrator calls the *mount* program specifying the remote directory, the local directory on which it is to be mounted, and other information. The *mount* program parses the name of the remote directory to be mounted and discovers the name of the machine on which the remote directory is located. It then contacts that machine asking for a file handle for the remote directory. If the directory exists and is available for remote mounting, the server returns a file handle for the directory. Finally, it makes a MOUNT system call, passing the handle to the kernel.

The kernel then constructs a v-node for the remote directory and asks the

NFS client code in Fig. 5-14 to create an **r-node** (**remote i-node**) in its internal tables to hold the file handle. The v-node points to the r-node. Each v-node in the VFS layer will ultimately contain either a pointer to an r-node in the NFS client code, or a pointer to an i-node in the local operating system (see Fig. 5-14). Thus from the v-node it is possible to see if a file or directory is local or remote, and if it is remote, to find its file handle.

When a remote file is opened, at some point during the parsing of the path name, the kernel hits the directory on which the remote file system is mounted. It sees that this directory is remote and in the directory's v-node finds the pointer to the r-node. It then asks the NFS client code to open the file. The NFS client code looks up the remaining portion of the path name on the remote server associated with the mounted directory and gets back a file handle for it. It makes an r-node for the remote file in its tables and reports back to the VFS layer, which puts in its tables a v-node for the file that points to the r-node. Again here we see that every open file or directory has a v-node that points to either an r-node or an i-node.

The caller is given a file descriptor for the remote file. This file descriptor is mapped onto the v-node by tables in the VFS layer. Note that no table entries are made on the server side. Although the server is prepared to provide file handles upon request, it does not keep track of which files happen to have file handles outstanding and which do not. When a file handle is sent to it for file access, it checks the handle, and if it is valid, uses it. Validation can include verifying an authentication key contained in the RPC headers, if security is enabled.

When the file descriptor is used in a subsequent system call, for example, READ, the VFS layer locates the corresponding v-node, and from that determines whether it is local or remote and also which i-node or r-node describes it.

For efficiency reasons, transfers between client and server are done in large chunks, normally 8192 bytes, even if fewer bytes are requested. After the client's VFS layer has gotten the 8K chunk it needs, it automatically issues a request for the next chunk, so it will have it should it be needed shortly. This feature, known as **read ahead**, improves performance considerably.

For writes an analogous policy is followed. If a WRITE system call supplies fewer than 8192 bytes of data, the data are just accumulated locally. Only when the entire 8K chunk is full is it sent to the server. However, when a file is closed, all of its data are sent to the server immediately.

Another technique used to improve performance is caching, as in ordinary UNIX. Servers cache data to avoid disk accesses, but this is invisible to the clients. Clients maintain two caches, one for file attributes (i-nodes) and one for file data. When either an i-node or a file block is needed, a check is made to see if it can be satisfied out of the cache. If so, network traffic can be avoided.

While client caching helps performance enormously, it also introduces some

nasty problems. Suppose that two clients are both caching the same file block and that one of them modifies it. When the other one reads the block, it gets the old (stale) value. The cache is not coherent. We saw the same problem with multiprocessors earlier. However, there it was solved by having the caches snoop on the bus to detect all writes and invalidate or update cache entries accordingly. With a file cache that is not possible, because a write to a file that results in a cache hit on one client does not generate any network traffic. Even if it did, snooping on the network is nearly impossible with current hardware.

Given the potential severity of this problem, the NFS implementation does several things to mitigate it. For one, associated with each cache block is a timer. When the timer expires, the entry is discarded. Normally, the timer is 3 sec for data blocks and 30 sec for directory blocks. Doing this reduces the risk somewhat. In addition, whenever a cached file is opened, a message is sent to the server to find out when the file was last modified. If the last modification occurred after the local copy was cached, the cache copy is discarded and the new copy fetched from the server. Finally, once every 30 sec a cache timer expires, and all the dirty (i.e., modified) blocks in the cache are sent to the server.

Still, NFS has been widely criticized for not implementing the proper UNIX semantics. A write to a file on one client may or may not be seen when another client reads the file, depending on the timing. Furthermore, when a file is created, it may not be visible to the outside world for as much as 30 sec. Similar problems exist as well.

From this example we see that although NFS provides a shared file system, because the resulting system is kind of a patched-up UNIX, the semantics of file access are not entirely well defined, and running a set of cooperating programs again may give different results, depending on the timing. Furthermore, the only issue NFS deals with is the file system. Other issues, such as process execution, are not addressed at all. Nevertheless, NFS is popular and widely used.

5.2.6. Lessons Learned

Based on his experience with various distributed file systems, Satyanarayanan (1990b) has stated some general principles that he believes distributed file system designers should follow. We have summarized these in Fig. 5-15. The first principle says that workstations have enough CPU power that it is wise to use them wherever possible. In particular, given a choice of doing something on a workstation or on a server, choose the workstation because server cycles are precious and workstation cycles are not.

The second principle says to use caches. They can frequently save a large amount of computing time and network bandwidth.

The third principle says to exploit usage properties. For example, in a

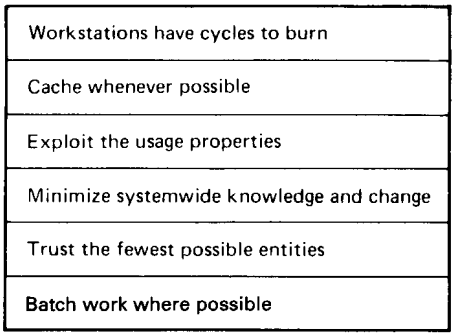


Fig. 5-15. Distributed file system design principles.

typical UNIX system, about a third of all file references are to temporary files, which have short lifetimes and are never shared. By treating these specially, considerable performance gains are possible. In all fairness, there is another school of thought that says: "Pick a single mechanism and stick to it. Do not have five ways of doing the same thing." Which view one takes depends on whether one prefers efficiency or simplicity.

Minimizing systemwide knowledge and change is important for making the system scale. Hierarchical designs help in this respect.

Trusting the fewest possible entities is a long-established principle in the security world. If the correct functioning of the system depends on 10,000 workstations all doing what they are supposed to, the system has a big problem.

Finally, batching can lead to major performance gains. Transmitting a 50K file in one blast is much more efficient than sending it as 50 1K blocks.

5.3. TRENDS IN DISTRIBUTED FILE SYSTEMS

Although rapid change has been a part of the computer industry since its inception, new developments seem to be coming faster than ever in recent years, both in the hardware and software areas. Many of these hardware changes are likely to have major impact on the distributed file systems of the future. In addition to all the improvements in the technology, changing user expectations and applications are also likely to have a major impact. In this section, we will survey some of the changes that can be expected in the foreseeable future and discuss some of the implications these changes may have for file systems. This section will raise more questions than it will answer, but it will suggest some interesting directions for future research.

5.3.1. New Hardware

Before looking at new hardware, let us look at old hardware with new prices. As memory continues to get cheaper and cheaper, we may see a revolution in the way file servers are organized. Currently, all file servers use magnetic disks for storage. Main memory is often used for server caching, but this is merely an optimization for better performance. It is not essential.

Within a few years, memory may become so cheap that even small organizations can afford to equip all their file servers with gigabytes of physical memory. As a consequence, the file system may permanently reside in memory, and no disks will be needed. Such a step will give a large gain in performance and will greatly simplify file system structure.

Most current file systems organize files as a collection of blocks, either as a tree (e.g., UNIX) or as a linked list (e.g., MS-DOS). With an in-core file system, it may be much simpler to store each file contiguously in memory, rather than breaking it up into blocks. Contiguously stored files are easier to keep track of and can be shipped over the network faster. The reason that contiguous files are not used on disk is that if a file grows, moving it to an area of the disk with more room is too expensive. In contrast, moving a file to another area of memory is feasible.

Main memory file servers introduce a serious problem, however. If the power fails, all the files are lost. Unlike disks, which do not lose information in a power failure, main memory is erased when the electricity is removed. The solution may be to make continuous or at least incremental backups onto videotape. With current technology, it is possible to store about 5 gigabytes on a single 8mm videotape that costs less than 10 dollars. While access time is long, if access is needed only once or twice a year to recover from power failures, this scheme may prove irresistible.

A hardware development that may affect file systems is the optical disk. Originally, these devices had the property that they could be written once (by burning holes in the surface with a laser), but not changed thereafter. They were sometimes referred to as **WORM** (**Write Once Read Many**) devices. Some current optical disks use lasers to affect the crystal structure of the disk, but do not damage them, so they can be erased.

Optical disks have three important properties:

1. They are slow.
2. They have huge storage capacities.
3. They have random access.

They are also relatively cheap, although more expensive than videotape. The

first two properties are the same as videotape, but the third opens the following possibility. Imagine a file server with an n-gigabyte file system in main memory, and an n-gigabyte optical disk as backup. When a file is created, it is stored in main memory and marked as not yet backed up. All accesses are done using main memory. When the workload is low, files that are not yet backed up are transferred to the optical disk in the background, with byte k in memory going to byte k on the disk. Like the first scheme, what we have here is a main memory file server, but with a more convenient backup device having a one-to-one mapping with the memory.

Another interesting hardware development is very fast fiber optic networks. As we discussed earlier, the reason for doing client caching, with all its inherent complications, is to avoid the slow transfer from the server to the client. But suppose that we could equip the system with a main memory file server and a fast fiber optic network. It might well become feasible to get rid of the client's cache and the server's disk and just operate out of the server's memory, backed up by optical disk. This would certainly simplify the software.

When studying client caching, we saw that a large fraction of the trouble is caused by the fact that if two clients are caching the same file and one of them modifies it, the other does not discover this, which leads to inconsistencies. A little thought will reveal that this situation is highly analogous to memory caches in a multiprocessor. Only there, when one processor modifies a shared word, a hardware signal is sent over the memory bus to the other caches to allow them to invalidate or update that word. With distributed file systems, this is not done.

Why not, actually? The reason is that current network interfaces do not support such signals. Nevertheless, it should be possible to build network interfaces that do. As a very simple example, consider the system of Fig. 5-16 in which each network interface has a bit map, one bit per cached file. To modify a file, a processor sets the corresponding bit in the interface, which is 0 if no processor is currently updating the file. Setting a bit causes the interface to create and send a packet around the ring that checks and sets the corresponding bit in all interfaces. If the packet makes it all the way around without finding any other machines trying to use the file, some other register in the interface is set to 1. Otherwise, it is set to 0. In effect, this mechanism provides a way to globally lock the file on all machines in a few microseconds.

After the lock has been set, the processor updates the file. Each block of the file that is changed is noted (e.g., using bits in the page table). When the update is complete, the processor clears the bit in the bit map, which causes the network interface to locate the file using a table in memory and automatically deposit all the modified blocks in their proper locations on the other machines. When the file has been updated everywhere, the bit in the bit map is cleared on all machines.

Clearly, this is a simple solution that can be improved in many ways, but it

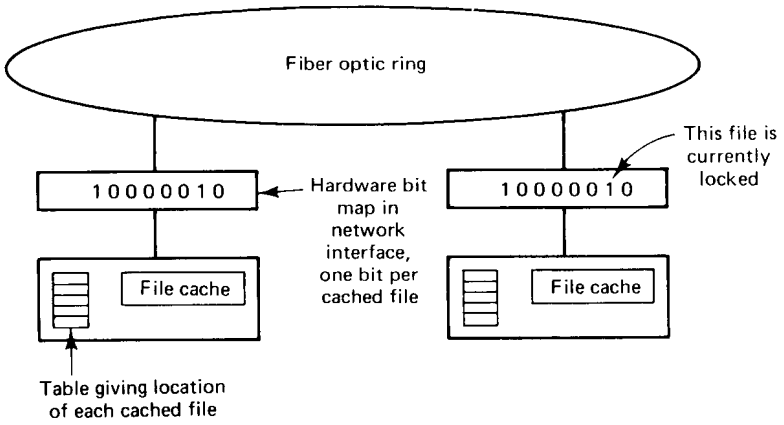


Fig. 5-16. A hardware scheme to updating shared files.

shows how a small amount of well-designed hardware can solve problems that are difficult to handle in software. It is likely that future distributed systems will be assisted by specialized hardware of various kinds.

5.3.2. Scalability

A definite trend in distributed systems is toward larger and larger systems. This observation has implications for distributed file system design. Algorithms that work well for systems with 100 machines may work poorly for systems with 1000 machines and not at all for systems with 10,000 machines. For starters, centralized algorithms do not scale well. If opening a file requires contacting a single centralized server to record the fact that the file is open, that server will eventually become a bottleneck as the system grows.

A general way to deal with this problem is to partition the system into smaller units and try to make each one relatively independent of the others. Having one server per unit scales much better than a single server. Even having the servers record all the opens may be acceptable under these circumstances.

Broadcasts are another problem area. If each machine issues one broadcast per second, with n machines, a total of n broadcasts per second appear on the network, generating a total of n^2 interrupts total. Obviously, as n grows, this will eventually be a problem.

Resources and algorithms should not be linear in the number of users, so having a server maintain a linear list of users for protection or other purposes is not a good idea. In contrast, hash tables are acceptable, since the access time is more or less constant, almost independent of the number of entries.

In general, strict semantics, such as UNIX semantics, get harder to implement as systems get bigger. Weaker guarantees are much easier to implement. Clearly, there is a trade-off here, since programmers prefer easily well-defined semantics, but these are precisely the ones that do not scale well.

In a very large system, the concept of a single UNIX-like file tree may have to be reexamined. It is inevitable that as the system grows, the length of path names will grow too, adding more overhead. At some point it may be necessary to partition the tree into smaller trees.

5.3.3. Wide Area Networking

Most current work on distributed systems focuses on LAN-based systems. In the future, many LAN-based distributed systems will be interconnected to form transparent distributed systems covering countries and continents. As an example, the French PTT is currently putting a small computer in every apartment and house in France. Although the initial goal is to eliminate the need for information operators and telephone books, at some point in time someone is going to ask if it is possible to connect 10 million or more computers spread over all of France into a single transparent system, for applications as yet undreamed of. What kind of file system would be needed to serve all of France? All of Europe? The entire world? At present, no one knows.

Although the French machines are all identical, in most wide-area networks, a large variety of equipment is encountered. This diversity is inevitable when multiple buyers with different-sized budgets and goals are involved, and the purchasing is spread over many years in an era of rapid technological change. Thus a wide-area distributed system must of necessity deal with heterogeneity. This raises issues such as how should you store a character file if not everyone uses ASCII, or what format one should use for files containing floating-point numbers if multiple representations are in use.

Also important is the expected change in applications. Most experimental distributed systems being built at universities focus on programming in a UNIX-like environment as the canonical application, because that is what the researchers themselves do all day (at least when they are not in committee meetings or writing grant proposals). Initial data suggest that not all 50 million French citizens are going to list C programming as their primary activity. As distributed systems become more widespread, we are likely to see a shift to electronic mail, electronic banking, accessing data bases, and recreational activities, which will change file usage, access patterns, and a great deal more in ways we as yet do not know.

An inherent problem with massive distributed systems is that the network bandwidth is extremely low. If the telephone line is the main connection, getting more than 64 Kbps out of it seems unlikely. Bringing fiber optics into

everyone's house will take decades and cost billions. On the other hand, vast amounts of data can be stored cheaply on compact disks and videotapes. Instead of logging into the telephone company's computer to look up a telephone number, it may be cheaper for them to send everyone a disk or tape containing the entire data base. We may have to develop file systems in which a distinction is made between static, read-only information (e.g., the phone book), and dynamic information (e.g., electronic mail). This distinction may have to become the basis of the entire file system.

5.3.4. Mobile Users

Portable computers are the fastest-growing segment of the computer business. Laptop computers, notebook computers, and pocket computers can be found everywhere these days, and they are multiplying like rabbits. Although computing while driving is hard, computing while flying is not. Telephones are now common in airplanes, so can flying FAXes and mobile modems be far behind? Nevertheless, the total bandwidth available from an airplane to the ground is quite low, and many places users want to go have no online connection at all.

The inevitable conclusion is that a large fraction of the time, the user will be off-line, disconnected from the file system. Few current systems were designed for such use, although Satyanarayanan (1990b) has reported some initial work in this direction.

Any solution is probably going to have to be based on caching. While connected, the user downloads to the portable those files expected to be needed later. These are used while disconnected. When reconnect occurs, the files in the cache will have to be merged with those in the file tree. Since disconnect can last for hours or days, the problems of maintaining cache consistency are much more severe than in online systems.

Another problem is that when reconnection does occur, the user may be in a city far away from his home base. Placing a phone call to the home machine is one way to get resynchronized, but the telephone bandwidth is low. Besides, in a truly distributed system contacting the local file server should be enough. The design of a worldwide, fully transparent distributed file system for simultaneous use by millions of mobile and frequently disconnected users is left as an exercise for the reader.

5.3.5. Fault Tolerance

Current computer systems, except for very specialized ones like air traffic control, are not fault tolerant. When the computer goes down, the users are expected to accept this as a fact of life. Unfortunately, the general population

expects things to work. If a television channel, the phone system, or the electric power company goes down for half an hour, there are many unhappy people the next day. As distributed systems become more and more widespread, the demand for systems that essentially never fail will grow. Current systems cannot meet this need.

Obviously, such systems will need considerable redundancy in hardware and the communication infrastructure, but they will also need it in software and especially data. File replication, often an afterthought in current distributed systems, will become an essential requirement in future ones. Systems will also have to be designed that manage to function when only partial data are available, since insisting that all the data be available all the time does not lead to fault tolerance. Down times that are now considered acceptable by programmers and other sophisticated users, will be increasingly unacceptable as computer use spreads to nonspecialists.

5.3.6. Multimedia

New applications, especially those involving real-time video or multimedia will have a large impact on future distributed file systems. Text files are rarely more than a few megabytes long, but video files can easily exceed a gigabyte. To handle applications such as video-on-demand, completely different file systems will be needed.

5.4. SUMMARY

The heart of any distributed system is the distributed file system. The design of such a file system begins with the interface: What is the model of a file, and what functionality is provided? As a rule, the nature of a file is no different for the distributed case than for the single-processor case. As usual, an important part of the interface is file naming and the directory system. Naming quickly brings up the issue of transparency. To what extent is the name of a file related to its location? Can the system move a single file on its own without the file name being affected? Different systems have different answers to these questions.

File sharing in a distributed system is a complex but important topic. Various semantic models have been proposed, including UNIX semantics, session semantics, immutable files, and transaction semantics. Each has its own strengths and weaknesses. UNIX semantics is intuitive and familiar to most programmers (even non-UNIX programmers), but it is expensive to implement. Session semantics is less deterministic, but more efficient. Immutable files are

unfamiliar to most people, and make updating files difficult. Transactions are frequently overkill.

Implementing a distributed file system involves making many decisions. These include whether the system should be stateless or stateful, if and how caching should be done, and how file replication can be managed. Each of these has far-ranging consequences for the designers and the users. NFS illustrates one way of building a distributed file system.

Future distributed file systems will probably have to deal with changes in hardware technology, scalability, wide-area systems, mobile users, and fault tolerance, as well as the introduction of multimedia. Many exciting challenges await us.

PROBLEMS

1. What is the difference between a file service using the upload/download model and one using the remote access model?
2. A file system allows links from one directory to another. In this way, a directory can "include" a subdirectory. In this context, what is the essential criterion that distinguishes a tree-structured directory system from a general graph-structured system?
3. In the text it was pointed out that shared file pointers cannot be implemented reasonably with session semantics. Can they be implemented when there is a single file server that provides UNIX semantics?
4. Name two useful properties that immutable files have.
5. Why do some distributed systems use two-level naming?
6. Why do stateless servers have to include a file offset in each request? Is this also needed for stateful servers?
7. One of the arguments given in the text in favor of stateful file servers is that i-nodes can be kept in memory for open files, thus reducing the number of disk operations. Propose an implementation for a stateless server that achieves almost the same performance gain. In what ways, if any, is your proposal better or worse than the stateful one?
8. When session semantics are used, it is always true that changes to a file are immediately visible to the process making the change and never visible to processes on other machines. However, it is an open question as to whether or not they should be immediately visible to other processes on the same machine. Give an argument each way.

9. Why can file caches use LRU whereas virtual memory paging algorithms cannot? Back up your arguments with approximate figures.

10. In the section on cache consistency, we discussed the problem of how a client cache manager knows if a file in its cache is still up-to-date. The method suggested was to contact the server and have the server compare the client and server times. Does this method fail if the client and server clocks are very different?

11. Consider a system that does client caching using the write-through algorithm. Individual blocks, rather than entire files, are cached. Suppose that a client is about to read an entire file sequentially, and some of the blocks are in the cache and others are not. What problem may occur, and what can be done about it?

12. Imagine that a distributed file uses client caching with a delayed write back policy. One machine opens, modifies, and closes a file. About half a minute later, another machine reads the file from the server. Which version does it get?

13. Some distributed file systems use client caching with delayed writes back to the server or write-on-close. In addition to the problems with the semantics, these systems introduce another problem. What is it? (*Hint*: Think about reliability.)

14. Measurements have shown that many files have an extremely short lifetime. What implication does this observation have for client caching policy?

15. Some distributed file systems use two-level names, ASCII and binary, as we have discussed throughout this chapter; others do not, and use ASCII names throughout. Similarly some file servers are stateless and some are stateful, giving four combinations of these two features. One of these combinations is somewhat less desirable than its alternatives. Which one, and why is it less desirable?

16. When file systems replicate files, they do not normally replicate all files. Give an example of a kind of file that is not worth replicating.

17. A file is replicated on 10 servers. List all the combinations of read quorum and write quorum that are permitted by the voting algorithm.

18. With a main memory file server that stores files contiguously, when a file grows beyond its current allocation unit, it will have to be copied. Suppose that the average file is 20K bytes and it takes 200 nsec to copy a 32-bit word. How many files can be copied per second? Can you suggest a way to do this copying without tying up the file server's CPU the entire time?

19. In NFS, when a file is opened, a file handle is returned, analogous to a file descriptor being returned in UNIX. Suppose that an NFS server crashes after a file handle has been given to a user. When the server reboots, will the file handle still be valid? If so, how does it work? If not, does this violate the principle of statelessness?

20. In the bit-map scheme of Fig. 5-16, is it necessary that all machines caching a given file use the same table entry for it? If so, how can this be arranged?

6

Distributed Shared Memory

In Chap. 1 we saw that two kinds of multiple-processor systems exist: multiprocessors and multicomputers. In a multiprocessor, two or more CPUs share a common main memory. Any process, on any processor, can read or write any word in the shared memory simply by moving data to or from the desired location. In a multicomputer, in contrast, each CPU has its own private memory. Nothing is shared.

To make an agricultural analogy, a multiprocessor is a system with a herd of pigs (processes) eating from a single feeding trough (shared memory). A multicomputer is a design in which each pig has its own private feeding trough. To make an educational analogy, a multiprocessor is a blackboard in the front of the room which all the students are looking at, whereas a multicomputer is each student looking at his or her own notebook. Although this difference may seem minor, it has far-reaching consequences.

The consequences affect both hardware and software. Let us first look at the implications for the hardware. Designing a machine in which many processors use the same memory simultaneously is surprisingly difficult. Bus-based multiprocessors, as described in Sec. 1.3.1, cannot be used with more than a few dozen processors because the bus tends to become a bottleneck. Switched multiprocessors, as described in Sec. 1.3.2, can be made to scale to large systems, but they are relatively expensive, slow, complex, and difficult to maintain.

In contrast, large multicomputers are easier to build. One can take an

almost unlimited number of single-board computers, each containing a CPU, memory, and a network interface, and connect them together. Multicomputers with thousands of processors are commercially available from various manufacturers. (Please note that throughout this chapter we use the terms "CPU" and "processor" interchangeably.) From a hardware designer's perspective, multicomputers are generally preferable to multiprocessors.

Now let us consider the software. Many techniques are known for programming multiprocessors. For communication, one process just writes data to memory, to be read by all the others. For synchronization, critical regions can be used, with semaphores or monitors providing the necessary mutual exclusion. There is an enormous body of literature available on interprocess communication and synchronization on shared-memory machines. Every operating systems textbook written in the past twenty years devotes one or more chapters to the subject. In short, a large amount of theoretical and practical knowledge exists about how to program a multiprocessor.

With multicomputers, the reverse is true. Communication generally has to use message passing, making input/output the central abstraction. Message passing brings with it many complicating issues, among them flow control, lost messages, buffering, and blocking. Although various solutions have been proposed, programming with message passing remains tricky.

To hide some of the difficulties associated with message passing, Birrell and Nelson (1984) proposed using remote procedure calls. In their scheme, now widely used, the actual communication is hidden away in library procedures. To use a remote service, a process just calls the appropriate library procedure, which packs the operation code and parameters into a message, sends it over the network, and waits for the reply. While this frequently works, it cannot easily be used to pass graphs and other complex data structures containing pointers. It also fails for programs that use global variables, and it makes passing large arrays expensive, since they must be passed by value rather than by reference.

In short, from a software designer's perspective, multiprocessors are definitely preferable to multicomputers. Herein lies the dilemma. Multicomputers are easier to build but harder to program. Multiprocessors are the opposite: harder to build but easier to program. What we need are systems that are both easy to build and easy to program. Attempts to build such systems form the subject of this chapter.

6.1. INTRODUCTION

In the early days of distributed computing, everyone implicitly assumed that programs on machines with no physically shared memory (i.e., multicomputers) obviously ran in different address spaces. Given this mindset, communication

was naturally viewed in terms of message passing between disjoint address spaces, as described above. In 1986, Li proposed a different scheme, now known under the name **distributed shared memory** (**DSM**) (Li, 1986; and Li and Hudak, 1989). Briefly summarized, Li and Hudak proposed having a collection of workstations connected by a LAN share a single paged, virtual address space. In the simplest variant, each page is present on exactly one machine. A reference to a *local* pages is done in hardware, at full memory speed. An attempt to reference a page on a different machine causes a hardware page fault, which traps to the operating system. The operating system then sends a message to the remote machine, which finds the needed page and sends it to the requesting processor. The faulting instruction is then restarted and can now complete.

In essence, this design is similar to traditional virtual memory systems: when a process touches a nonresident page, a trap occurs and the operating system fetches the page and maps it in. The difference here is that instead of getting the page from the disk, the operating system gets it from another processor over the network. To the user processes, however, the system looks very much like a traditional multiprocessor, with multiple processes free to read and write the shared memory at will. All communication and synchronization can be done via the memory, with no communication visible to the user processes. In effect, Li and Hudak devised a system that is both easy to program (logically shared memory) and easy to build (no physically shared memory).

Unfortunately, there is no such thing as a free lunch. While this system is indeed easy to program and easy to build, for many applications it exhibits poor performance, as pages are hurled back and forth across the network. This behavior is analogous to thrashing in single-processor virtual memory systems. In recent years, making these distributed shared memory systems more efficient has been an area of intense research, with numerous new techniques discovered. All of these have the goal of minimizing the network traffic and reducing the latency between the moment a memory request is made and the moment it is satisfied.

One approach is not to share the entire address space, only a selected portion of it, namely just those variables or data structures that need to be used by more than one process. In this model, one does not think of each machine as having direct access to an ordinary memory but rather, to a collection of shared variables, giving a higher level of abstraction. Not only does this strategy greatly reduce the amount of data that must be shared, but in most cases, considerable information about the shared data is available, such as their types, which can help optimize the implementation.

One possible optimization is to replicate the shared variables on multiple machines. By sharing replicated variables instead of entire pages, the problem of simulating a multiprocessor has been reduced to that of how to keep multiple

copies of a set of typed data structures consistent. Potentially, reads can be done locally without any network traffic, and writes can be done using a multicopy update protocol. Such protocols are widely used in distributed data base systems, so ideas from that field may be of use.

Going still further in the direction of structuring the address space, instead of just sharing variables we could share encapsulated data types, often called **objects**. These differ from shared variables in that each object has not only some data, but also procedures, called **methods**, that act on the data. Programs may only manipulate an object's data by invoking its methods. Direct access to the data is not permitted. By restricting access in this way, various new optimizations become possible.

Doing everything in software has a different set of advantages and disadvantages from using the paging hardware. In general, it tends to put more restrictions on the programmer but may achieve better performance. Many of these restrictions (e.g., working with objects) are considered good software engineering practice and are desirable in their own right. We will come back to this subject later.

Before getting into distributed shared memory in more detail, we must first take a few steps backward to see what shared memory really is and how shared-memory multiprocessors actually work. After that we will examine the semantics of sharing, since they are surprisingly subtle. Finally, we will come back to the design of distributed shared memory systems. Because distributed shared memory can be intimately related to computer architecture, operating systems, runtime systems, and even programming languages, all of these topics will come into play in this chapter.

6.2. WHAT IS SHARED MEMORY?

In this section we will examine several kinds of shared memory multiprocessors, ranging from simple ones that operate over a single bus, to advanced ones with highly sophisticated caching schemes. These machines are important for an understanding of distributed shared memory because much of the DSM work is being inspired by advances in multiprocessor architecture. Furthermore, many of the algorithms are so similar that it is sometimes difficult to tell whether an advanced machine is a multiprocessor or a multicomputer using a hardware implementation of distributed shared memory. We will conclude by comparing the various multiprocessor architectures to some distributed shared memory systems and discover that there is a spectrum of possible designs, from those entirely in hardware to those entirely in software. By examing the entire spectrum, we can get a better feel for where DSM fits in.

6.2.1. On-Chip Memory

Although most computers have an external memory, self-contained chips containing a CPU and all the memory also exist. Such chips are produced by the millions, and are widely used in cars, appliances, and even toys. In this design, the CPU portion of the chip has address and data lines that directly connect to the memory portion. Figure 6-1(a) shows a simplified diagram of such a chip.

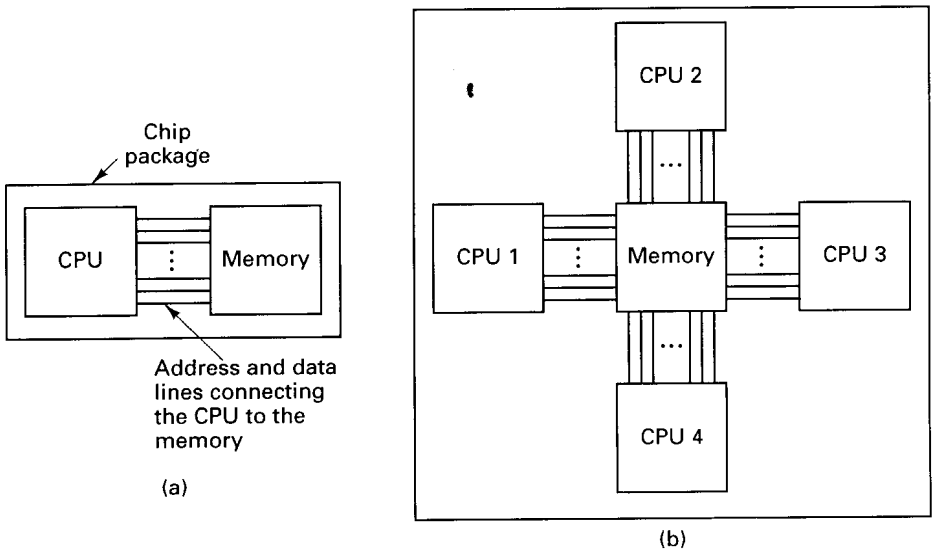


Fig. 6-1. (a) A single-chip computer. (b) A hypothetical shared-memory multiprocessor.

One could imagine a simple extension of this chip to have multiple CPUs directly sharing the same memory, as shown in Fig. 6-1(b). While it is possible to construct a chip like this, it would be complicated, expensive, and highly unusual. An attempt to construct a one-chip multiprocessor this way, with, say, 100 CPUs directly accessing the same memory would be impossible for engineering reasons. A different approach to sharing memory is needed.

6.2.2. Bus-Based Multiprocessors

If we look closely at Fig. 6-1(a), we see that the connection between the CPU and the memory is a collection of parallel wires, some holding the address the CPU wants to read or write, some for sending or receiving data, and the rest for controlling the transfers. Such a collection of wires is called a **bus**. This bus is on-chip, but in most systems, buses are external and are used to connect printed circuit boards containing CPUs, memories, and I/O controllers. On a

desktop computer, the bus is typically etched onto the main board (the parentboard), which holds the CPU and some of the memory, and into which I/O cards are plugged. On minicomputers the bus is sometimes a flat cable that wends its way among the processors, memories, and I/O controllers.

A simple but practical way to build a multiprocessor is to base it on a bus to which more than one CPU is connected. Fig. 6-2(a) illustrates a system with three CPUs and a memory shared among all of them. When any of the CPUs wants to read a word from the memory, it puts the address of the word it wants on the bus and asserts (puts a signal on) a bus control line indicating that it wants to do a read. When the memory has fetched the requested word, it puts the word on the bus and asserts another control line to announce that it is ready. The CPU then reads in the word. Writes work in an analogous way.

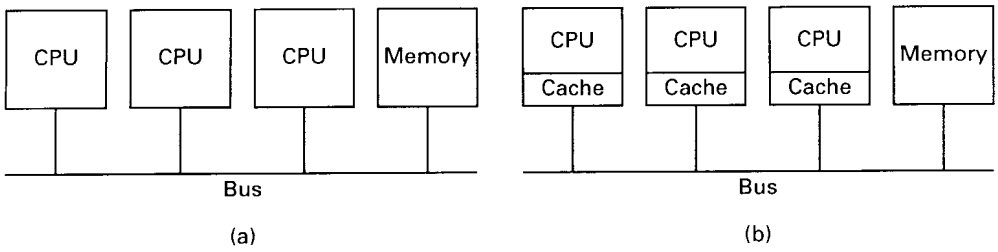


Fig. 6-2. (a) A multiprocessor. (b) A multiprocessor with caching.

To prevent two or more CPUs from trying to access the memory at the same time, some kind of bus arbitration is needed. Various schemes are in use. For example, to acquire the bus, a CPU might first have to request it by asserting a special request line. Only after receiving permission would it be allowed to use the bus. The granting of this permission can be done in a centralized way, using a bus arbitration device, or in a decentralized way, with the first requesting CPU along the bus winning any conflict.

The disadvantage of having a single bus is that with as few as three or four CPUs the bus is likely to become overloaded. The usual approach taken to reduce the bus load is to equip each CPU with a **snooping cache** (sometimes called a **snoopy cache),** so called because it "snoops" on the bus. Caches are shown in Fig. 6-2(b). They have been the subject of a large amount of research over the years (Agarwal et al., 1988; Agarwal and Cherian, 1989; Archibald and Baer, 1986; Cheong and Veidenbaum, 1988; Dahlgren et al., 1994; Eggers and Katz, 1989a, 1989b; Nayfeh and Olukotun, 1994; Przybylski et al., 1988; Scheurich and Dubois, 1987; Thekkath and Eggers, 1994; Vernon et al., 1988; and Weber and Gupta, 1989). All of these papers present slightly different **cache consistency protocols**, that is, rules for making sure that different caches do not contain different values for the same memory location.

One particularly simple and common protocol is called **write through**. When a CPU first reads a word from memory, that word is fetched over the bus and is stored in the cache of the CPU making the request. If that word is needed again later, the CPU can take it from the cache without making a memory request, thus reducing bus traffic. These two cases, read miss (word not cached) and read hit (word cached) are shown in Fig. 6-3 as the first two lines in the table. In simple systems, only the word requested is cached, but in most, a block of words of say, 16 or 32 words, is transferred and cached on the initial access and kept for possible future use.

Event	Action taken by a cache in response to its own CPU's operation	Action taken by a cache in response to a remote CPU's operation
Read miss	Fetch data from memory and store in cache	(No action)
Read hit	Fetch data from local cache	(No action)
Write miss	Update data in memory and store in cache	(No action)
Write hit	Update memory and cache	Invalidate cache entry

Fig. 6-3. The *write-through* cache consistency protocol. The entries for *hit* in the third column mean that the snooping CPU has the word in its cache, not that the requesting CPU has it.

Each CPU does its caching independent of the others. Consequently, it is possible for a particular word to be cached at two or more CPUs at the same time. Now let us consider what happens when a write is done. If no CPU has the word being written in its cache, the memory is just updated, as if caching were not being used. This operation requires a normal bus cycle. If the CPU doing the write has the only copy of the word, its cache is updated and memory is updated over the bus as well.

So far, so good. The trouble arises when a CPU wants to write a word that two or more CPUs have in their caches. If the word is currently in the cache of the CPU doing the write, the cache entry is updated. Whether it is or not, it is also written to the bus to update memory. All the other caches see the write (because they are snooping on the bus) and check to see if they are also holding the word being modified. If so, they invalidate their cache entries, so that after the write completes, memory is up-to-date and only one machine has the word in its cache.

An alternative to invalidating other cache entries is to update all of them. Updating is slower than invalidating in most cases, however. Invalidating requires supplying just the address to be invalidated, whereas updating needs to provide the new cache entry as well. If these two items must be presented on the bus consecutively, extra cycles will be required. Even if it is possible to put an address and a data word on the bus simultaneously, if the cache block size is

more than one word, multiple bus cycles will be needed to update the entire block. The issue of invalidate vs. update occurs in all cache protocols and also in DSM systems.

The complete protocol is summarized in Fig. 6-3. The first column lists the four basic events that can happen. The second one tells what a cache does in response to its *own* CPU's actions. The third one tells what happens when a cache sees (by snooping) that a *different* CPU has had a hit or miss. The only time cache S (the snooper) must do something is when it sees that another CPU has written a word that S has cached (a write hit from S's point of view). The action is for S to delete the word from its cache.

The *write-through* protocol is simple to understand and implement but has the serious disadvantage that all writes use the bus. While the protocol certainly reduces bus traffic to some extent, the number of CPUs that can be attached to a single bus is still too small to permit large-scale multiprocessors to be built using it.

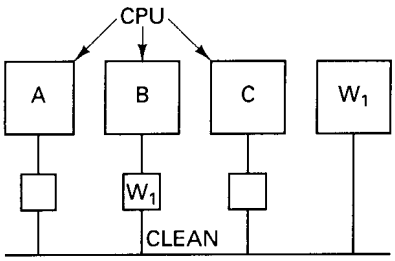
Fortunately, for many actual programs, once a CPU has written a word, that CPU is likely to need the word again, and it is unlikely that another CPU will use the word quickly. This situation suggests that if the CPU using the word could somehow be given temporary "ownership" of the word, it could avoid having to update memory on subsequent writes until a different CPU exhibited interest in the word. Such cache protocols exist. Goodman (1983) devised the first one, called **write once**. However, this protocol was designed to work with an existing bus and was therefore more complicated than is strictly necessary. Below we will describe a simplified version of it, which is typical of all ownership protocols. Other protocols are described and compared by Archibald and Baer (1986).

Our protocol manages cache blocks, each of which can be in one of the following three states:

1. INVALID — This cache block does not contain valid data.
2. CLEAN — Memory is up-to-date; the block may be in other caches.
3. DIRTY — Memory is incorrect; no other cache holds the block.

The basic idea is that a word that is being read by multiple CPUs is allowed to be present in all their caches. A word that is being heavily written by only one machine is kept in its cache and not written back to memory on every write to reduce bus traffic.

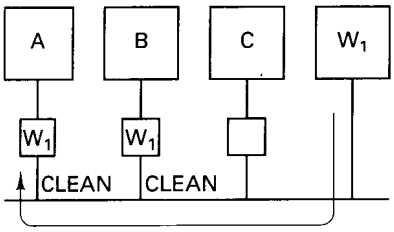
The operation of the protocol can best be illustrated by an example. For simplicity in this example, we will assume that each cache block consists of a single word. Initially, B has a cached copy of the word at address W, as illustrated in Fig. 6-4(a). The value is W_1. The memory also has a valid copy. In



Memory is correct

Initial state.
Word W containing value W_1 is in memory and is also cached by B.

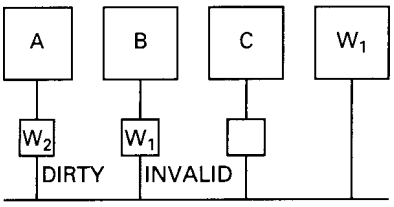
(a)



Memory is correct

A reads word W and gets W_1. B does not respond to the read, but the memory does.

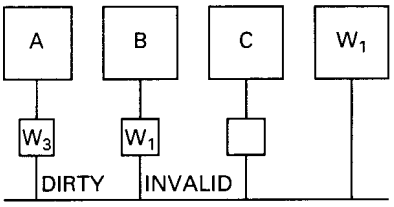
(b)



Memory is correct

A writes a value W_2. B snoops on the bus, sees the write, and invalidates its entry. A's copy is marked DIRTY.

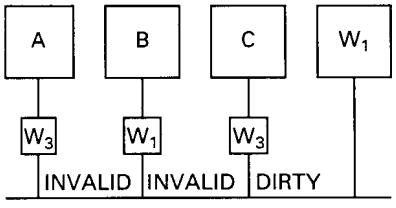
(c)



Memory is correct

A writes W again. This and subsequent writes by A are done locally, without any bus traffic.

(d)



Memory is correct

C reads or writes W. A sees the request by snooping on the bus, provides the value, and invalidates its own entry. C now has the only valid copy.

(e)

Fig. 6-4. An example of how a cache ownership protocol works.

Fig. 6-4(b), *A* requests and gets a copy of *W* from the memory. Although *B* sees the read request go by, it does not respond to it.

Now *A* writes a new value, W_2 to *W*. *B* sees the write request and responds by invalidating its cache entry. *A*'s state is changed to DIRTY, as shown in Fig. 6-4(c). The DIRTY state means that *A* has the only cached copy of *W* and that memory is out-of-date for *W*.

At this point, *A* overwrites the word again, as shown in Fig. 6-4(d). The write is done locally, in the cache, with no bus traffic. All subsequent writes also avoid updating memory.

Sooner or later, some other CPU, *C* in Fig. 6-4(e), accesses the word. *A* sees the request on the bus and asserts a signal that inhibits memory from responding. Instead, *A* provides the needed word and invalidates its own entry. *C* sees that the word is coming from another cache, not from memory, and that it is in DIRTY state, so it marks the entry accordingly. *C* is now the owner, which means that it can now read and write the word without making bus requests. However, it also has the responsibility of watching out for other CPUs that request the word, and servicing them itself. The word remains in DIRTY state until it is purged from the cache it is currently residing in for lack of space. At that time it disappears from all caches and is written back to memory.

Many small multiprocessors use a cache consistency protocol similar to this one, often with small variations. It has three important properties:

1. Consistency is achieved by having all the caches do bus snooping.
2. The protocol is built into the memory management unit.
3. The entire algorithm is performed in well under a memory cycle.

As we will see later, some of these do not hold for larger (switched) multiprocessors, and none of them hold for distributed shared memory.

6.2.3. Ring-Based Multiprocessors

The next step along the path toward distributed shared memory systems are ring-based multiprocessors, exemplified by **Memnet** (Delp, 1988; Delp et al., 1991; and Tam et al., 1990). In Memnet, a single address space is divided into a private part and a shared part. The private part is divided up into regions so that each machine has a piece for its stacks and other unshared data and code. The shared part is common to all machines (and distributed over them) and is kept consistent by a hardware protocol roughly similar to those used on bus-based multiprocessors. Shared memory is divided into 32-byte blocks, which is the unit in which transfers between machines take place.

All the machines in Memnet are connected together in a modified token-

passing ring. The ring consists of 20 parallel wires, which together allow 16 data bits and 4 control bits to be sent every 100 nsec, for a data rate of 160 Mbps. The ring is illustrated in Fig. 6-5(a). The ring interface, MMU (Memory Management Unit), cache, and part of the memory are integrated together in the **Memnet device**, which is shown in the top third of Fig. 6-5(b).

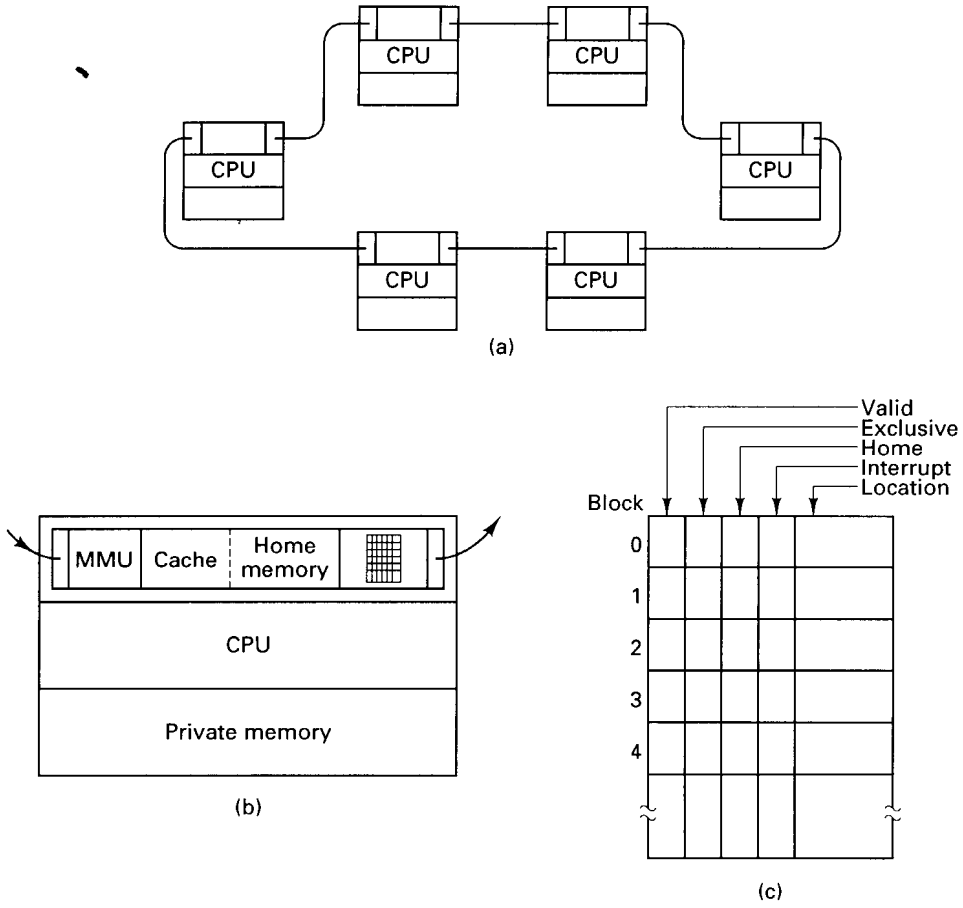


Fig. 6-5. (a) The Memnet ring. (b) A single machine. (c) The block table.

Unlike the bus-based multiprocessors of Fig. 6-2, in Memnet there is no centralized global memory. Instead, each 32-byte block in the shared address space has a home machine on which physical memory is always reserved for it, in the *Home memory* field of Fig. 6-5(b). A block may be cached on a machine other than its home machine. (The cache and home memory areas share the same buffer pool, but since they are used slightly differently, we treat them here as separate entities.) A read-only block may be present on multiple machines; a

read-write block may be present on only one machine. In both cases, a block need not be present on its home machine. All the home machine does is provide a guaranteed place to store the block if no other machine wants to cache it. This feature is needed because there is no global memory. In effect, the global memory has been spread out over all the machines.

The Memnet device on each machine contains a table, shown in Fig. 6-5(c), which contains an entry for each block in the shared address space, indexed by block number. Each entry contains a *Valid* bit telling whether the block is present in the cache and up to date, an *Exclusive* bit, specifying whether the local copy, if any, is the only one, a *Home* bit, which is set only if this is the block's home machine, an *Interrupt* bit, used for forcing interrupts, and a *Location* field that tells where the block is located in the cache if it is present and valid.

Having looked at the architecture of Memnet, let us now examine the protocols it uses. When the CPU wants to read a word from shared memory, the memory address to be read is passed to the Memnet device, which checks the block table to see if the block is present. If so, the request is satisfied immediately. If not, the Memnet device waits until it captures the circulating token, then puts a request packet onto the ring and suspends the CPU. The request packet contains the desired address and a 32-byte dummy field.

As the packet passes around the ring, each Memnet device along the way checks to see if it has the block needed. If so, it puts the block in the dummy field and modifies the packet header to inhibit subsequent machines from doing so. If the block's *Exclusive* bit is set, it is cleared. Because the block has to be somewhere, when the packet comes back to the sender, it is guaranteed to contain the block requested. The CPU sending the request then stores the block, satisfies the request, and releases the CPU.

A problem arises if the requesting machine has no free space in its cache to hold the incoming block. To make space, it picks a cached block at random and sends it home, thus freeing up a cache slot. Blocks whose *Home* bit are set are never chosen since they are already home.

Writes work slightly differently than reads. Three cases have to be distinguished. If the block containing the word to be written is present and is the only copy in the system (i.e., the *Exclusive* bit is set), the word is just written locally.

If the needed block is present but it is not the only copy, an invalidation packet is first sent around the ring to force all other machines to discard their copies of the block about to be written. When the invalidation packet arrives back at the sender, the *Exclusive* bit is set for that block and the write proceeds locally.

If the block is not present, a packet is sent out that combines a read request and an invalidation request. The first machine that has the block copies it into